

$$A \rightarrow x_1 x_2 \dots x_n$$

If x_1 is a nonterminal, we have reached another choice point, based on x_1 , and therefore go to step 3. Otherwise, x_1 must match the left-most exposed input character. If it fails to match, we block and retreat to the backtrack move step 2. Each of the tokens $x_2, x_3, \dots, x_n$ are similarly examined, until we either match all of them or we find the left-most nonterminal.

Suppose all of them match. We then search the partially constructed tree for a left-most exposed node. (This procedure will require an algorithm for moving up to the parent, seeing if it has any exposed right siblings, etc., the details of which will not concern us here). If a left-most exposed node exists, and is nonterminal, go to step 3 (a choice of production is needed).

If a left-most exposed node exists and is terminal, it must match the left-most exposed input token. On a failure to match, go to step 2 (backtrack). On a match success, continue matching.

Finally, suppose that no left-most exposed node exists. Now either the input string is completely attached to the tree or not. If it is attached, then we halt and report "success". Otherwise, we block and go to step 2.

Step 2: The backtrack move. On any backtrack move, we must discard a portion of the partially complete tree and try another production at the left-most exposed nonterminal node. We need the two items of information in a tape cell: the left-most exposed nonterminal node and the particular production chosen for it. Given the node, we can delete the subtree hanging from it and determine the input string position. Given the production choice, we can decide if another production choice exists. If it does, take it, record it, and go to step 1. If another choice does not exist, go to step 2 again.

Step 3: The choice move. The choice move is easy: we record the current left-most exposed nonterminal tree node, then select and record a production compatible with that node; it should be the first of an ordered set of compatible productions. (If the exposed node is associated with nonterminal A , then the compatible productions are all those with the form $A \rightarrow w$.)

Example. A grammar for which the backtracking system will work is the following (we shall later demonstrate two other ways to parse sentences derivable in this grammar). This grammar describes decimal numbers (containing a decimal point) with an optional sign.

Grammar $G_3 = (\{V, S, R, N\}, \{+, -, ., d, \perp\}, P, V)$, where P is

1. $V \rightarrow SR\perp$ $\{\perp \text{ is a stop symbol}\}$
2. $S \rightarrow +$

3. $S \rightarrow -$
4. $S \rightarrow \epsilon$ $\{\epsilon \text{ is the empty string}\}$
5. $R \rightarrow .dN$ $\{d \text{ is a decimal digit}\}$
6. $R \rightarrow dN.N$
7. $N \rightarrow dN$
8. $N \rightarrow \epsilon$

Consider the input string:

$+.dd \perp$

A complete trace is shown in figure 2.15. There are no backups required for this string until the state shown in part (f) is reached. In trying the first choice ($N \rightarrow dN$) for the exposed node 9, we find that “d” and the next symbol $\perp$ fail to match, hence we must back up. The top cell says node 9 was given production 7 previously. Production 8 ($N \rightarrow \epsilon$) is still available, so we try it and find that we can match the remaining exposed node (the token $\perp$) and exhaust the input string.

Exercises

1. Trace the backtracking system on the strings

$dd.d \perp$
 $- \quad dd \perp$

2. Show that the backtracking system cannot accept either of the strings

$d- \perp$
 $- \quad + \perp$

3. Consider the following ambiguous grammar:

$S \rightarrow E \perp$
 $E \rightarrow aEE \mid \epsilon$

and its backtracking parser. Trace its behavior on the string

$aa \perp$

Discuss informally the factors influencing its choice of several possible parses.

Limitations of Backtracking

This system will succeed if and only if no left-recursive derivation in the basis grammar exists. A left-recursive derivation is such that

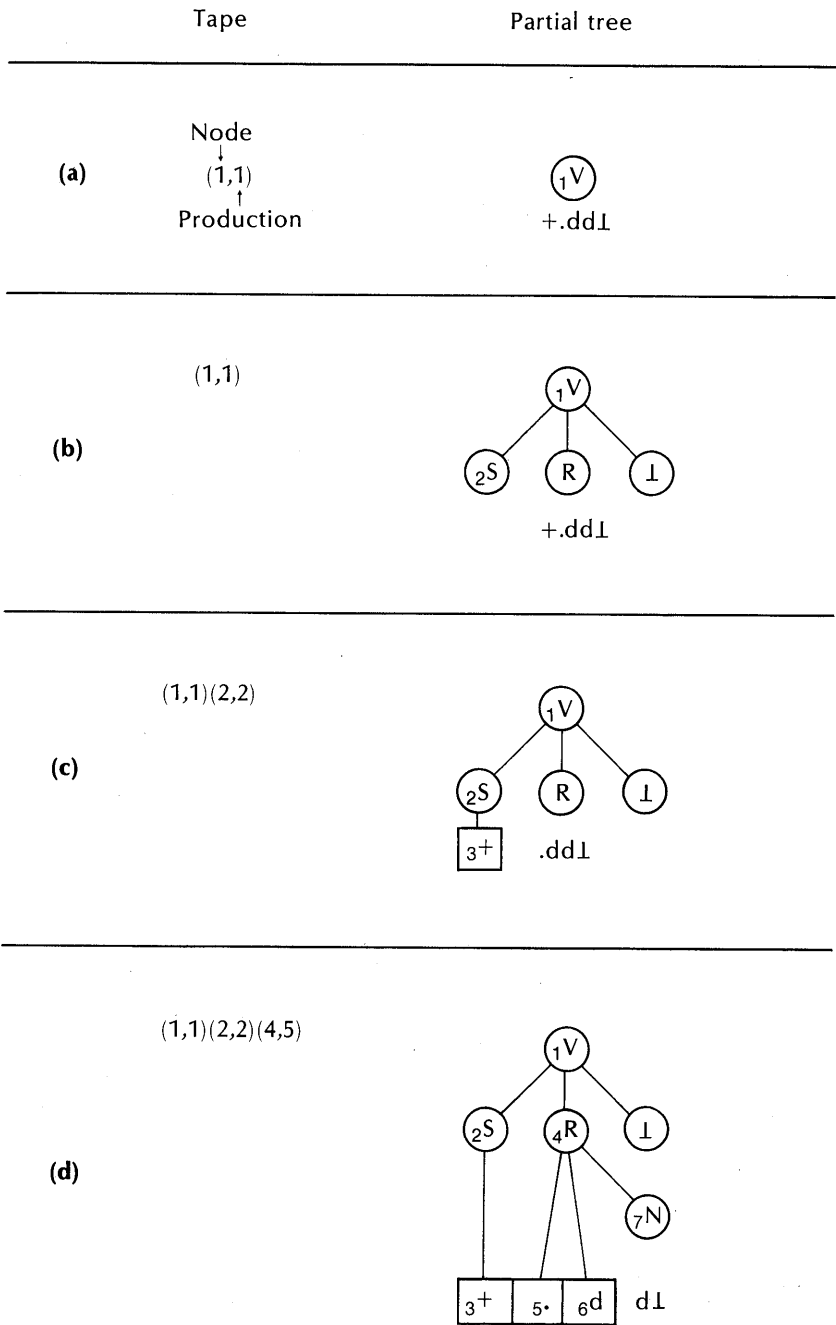
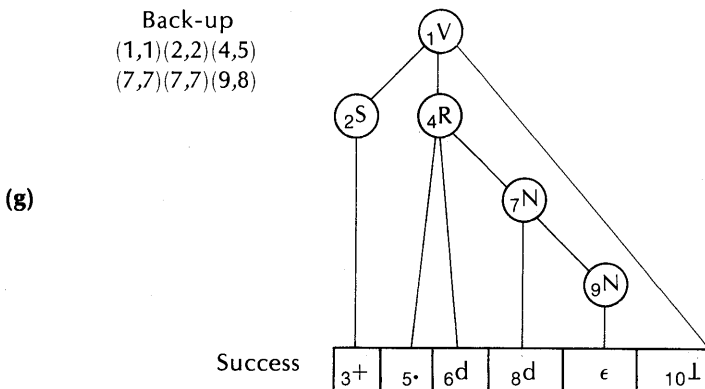
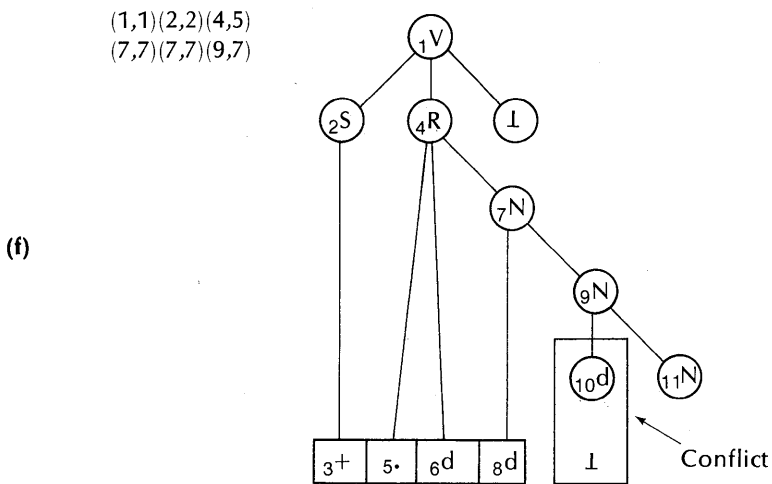
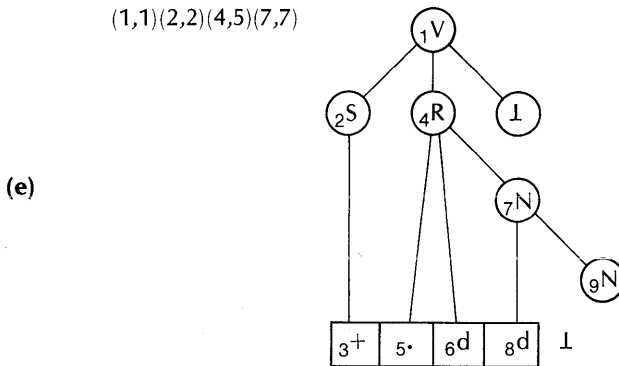


Figure 2.15. Trace of a top-down backtracking parse of a string “+.ddL” in grammar G_3 .



$$A \Rightarrow^+ Aw$$

for some nonterminal A . We shall not prove this; however, we can easily show that a left-recursion will cause a system failure. Consider grammar G_0 and the partial tree of figure 2.11. We have two productions compatible with the left-most exposed node T : $T \rightarrow F$ and $T \rightarrow T*F$. Note that the latter production is left-recursive. If we choose $T \rightarrow T*F$, we end up with another tree with the same exposed input string and the left-most exposed node T . The system will again choose $T \rightarrow T*F$ (assuming this is the first choice), etc. The tree and the backup tape will continue to grow indefinitely, with no progress in scanning the input string.

We might argue that the trouble lies in choosing $T \rightarrow T*F$ first. Why not arrange the productions so that left-recursive productions are chosen after the others? This choice seems to work in figure 2.11. We can add $T \rightarrow F$ to the tree and can continue nicely for awhile. In fact, by placing the left-recursive productions last among the choices, we can parse every string in the language $L(G_0)$.

However, this is not good enough. A parser must also be able to detect and report errors in syntax, i.e., it must be able to determine that some strings are not in the language. For example, suppose that in figure 2.11, the left-most exposed input string element were “*” instead of “(”. We have a string that is obviously not in the language, and there is therefore no subtree that can be attached to the node T that will match a “*”. What will the backtracking system do? It will attempt all possibilities. The $T \rightarrow F$ choice will eventually be found to fail (after many trials and errors), hence the choice $T \rightarrow T*F$ will be attempted. This choice, too, must fail because we eventually must get the same exposed input string and exposed nonterminal node as before and the system will run forever. We conclude that with a left-recursive production in the grammar, there are strings for which the system will never halt.

Aho [1972] shows that the backtracking system will never fail if the grammar is not left-recursive.

Time Bound

It can be shown (Aho [1972], chapter 4) that a parse of a string of length $n \geq 1$ for a non-left-recursive grammar will require no more than c^n operations. The c is some constant, > 1 , that is characteristic of the grammar. This is a “best” bound, to the extent that we restrict the grammar in no way other than requiring that it be non-left-recursive.

Indeed, there exist grammars that cause the backtracking system to spend a

time proportional to an exponential of the length of the input. For example, consider

$$\begin{aligned} S &\rightarrow cSS \\ S &\rightarrow \epsilon \end{aligned}$$

which derives sequences of c 's. If $Y(n)$ is the number of partial left parses consistent with string $w = cccc \dots c$, where $|w| = n$, then $Y(n)$ is certainly greater than 2^n (Aho [1972]).

This grammar causes the backtracking parser to construct and discard every possible partial tree before reaching its goal.

The behavior of any backtracking parser upon encountering a syntax error is also exponential in character. Since every possible partial tree consistent with the input string up to the position of the error must result in a block, the parser constructs and discards all of them. For a reasonably large grammar and strings of practical length, the time spent in such analysis is enormous.

We conclude that a backtracking parser system is impractical. We shall see that backtracking is unnecessary for a large class of grammars. There also exist more powerful generalized parsing methods (e.g., Earley [1968], also described in Aho [1972], chapter 4) that not only will parse any context-free grammar but do so with a better time bound than any backtracking system.

Exercises

1. Trace the backtracking parser for G_0 on the invalid string

$a^{**}\perp$

far enough to show that it will never halt. Use the left-recursive productions last.

2. Trace the backtracking parser on the grammar

$$S \rightarrow cSS \mid \epsilon$$

for strings

c
 cc
 ccc

and discuss the parsing pattern it exhibits.

2.3.3. A Deterministic Top-Down Parser

The backtracking parser of section 2.3.1 is said to be *nondeterministic*. That is, given a choice at some node in the partially constructed tree, it simply makes an arbitrary choice and prepares for the possibility (the very likely possibility!) that its choice will be wrong.

Suppose that we had some way of making the correct choice each time. For a top-down parser, we have some information in the exposed input string that could be used to make the correct choice. For example, in figure 2.11, the exposed input string is $(a*a)$, and this should be sufficient to determine that the correct choice of a T production is $T \rightarrow F$. We can then conceive of a large table such that each row is associated with a nonterminal node and each column with some legal input string. The table will then tell us which of several possible productions to choose for the next top-down move.

Unfortunately, such a table would be infinitely large—for interesting grammars, the number of possible unexposed input strings is infinite. For a practical compiler, we need a finite table.

Suppose instead we settle for a table such that every column contains only one input token, the left-most exposed string token, or *next token*. We clearly have potentially useful information in the rest of the exposed input string, but we can't use more than a finite amount of it anyway.

We still require that our table (now finite) fix a production choice for every possible situation. This requirement imposes a restriction on the basis grammar. It is possible to build such a table for some grammars and not for others.

Let us again consider grammar G_3 , introduced in the previous section.

Grammar $G_3 = (\{V, S, R, N\}, \{+, -, ., d, \perp\}, P, V)$, where P is:

1. $V \rightarrow SR\perp$ $\{\perp \text{ is a stop symbol}\}$
2. $S \rightarrow +$
3. $S \rightarrow -$
4. $S \rightarrow \epsilon$ $\{\epsilon \text{ is the empty string}\}$
5. $R \rightarrow .dN$ $\{d \text{ is a decimal digit}\}$
6. $R \rightarrow dN.N$
7. $N \rightarrow dN$
8. $N \rightarrow \epsilon$

A top-down, one-symbol parsing table can be constructed for this grammar, by methods described in chapter 4. It is given in figure 2.16. Each row corresponds to a possible exposed left-most nonterminal node in the partially constructed tree. Each column corresponds to the next token. The entries are either a production number (1 through 8) or an X. The X means that there must be a syntax error; there is no way that a derivation based on the exposed nonterminal can match that token. For example, with token “.”

		Next token				
		+	-	.	d	⊥
Left-most Exposed Nonterminal	V	1	1	1	1	×
	S	2	3	4	4	×
	R	×	×	5	6	×
	N	×	×	8	7	8

Figure 2.16. A top-down LL(1) parsing table for grammar G_3 .

and nonterminal S, the table says that production 4 ($S \rightarrow \epsilon$) is the appropriate one to attach to the tree.

We can illustrate a parse without drawing a lot of trees. All we really need is the frontier of the partially constructed tree and the remainder of the input string. Thus figure 2.11 has the frontier $a + T$ and the remaining string $(a*a)$.

Given these two strings, we apply the table to the left-most nonterminal in the frontier and the first token of the remaining string, which yields a replacement string w . If the first tokens in w are terminal tokens, they must either match the tokens in the input string or else a syntax error exists. If they match, we drop the matched tokens before applying the table again.

Let us trace the process with the string $-ddd.dd\perp$.

Frontier	Remaining Input	Production
V	$-ddd.dd\perp$	1
SR $\perp$	$-ddd.dd\perp$	3
$-R\perp$	$-ddd.dd\perp$	(match, drop “-”)
R $\perp$	$ddd.dd\perp$	6
dN.N $\perp$	$ddd.dd\perp$	(match)
N.N $\perp$	$dd.dd\perp$	7
dN.N $\perp$	$dd.dd\perp$	(match)
N.N $\perp$	$d.dd\perp$	7
dN.N $\perp$	$d.dd\perp$	(match)
N.N $\perp$	$.dd\perp$	8
.N $\perp$	$.dd\perp$	(match)
N $\perp$	$dd\perp$	7
dN $\perp$	$dd\perp$	(match)
N $\perp$	$d\perp$	7
dN $\perp$	$d\perp$	(match)
N $\perp$	$\perp$	8
$\perp$	$\perp$	(match and halt)

We stop and report “success” when both the tree frontier and the input list are empty. Other possibilities exist on input strings that are not in the language; for these the machine must report “failure.” We require that every input string be terminated with the special symbol $\perp$, and that this symbol not appear elsewhere in the input.

It can be shown that this parsing process has a time bound linear with the length of the input string, obviously a vast improvement over the backtracking approach of the previous section. However, we have paid for this time improvement with a certain restriction in the class of grammars that are amenable to this approach.

This parsing system is called an *LL(1) parser*, and was first described in Rosenkrantz [1970]. It has been used as the basis of several compilers, for example, see Lewis [1968]. “LL” means “Left-to-right, Left-most.” The “1” refers to the single input symbol used to resolve the production choice. We could also use 2, 3, . . . symbols, yielding an LL(2), LL(3), . . . parser.

Exercise

1. Trace the parser of figure 2.16 on the strings

dd.d $\perp$	
.ddd $\perp$	
-.d $\perp$	
-.+d $\perp$	{ illegal }
d.d+ $\perp$	{ illegal }
d.d. $\perp$	{ illegal }

2.3.4. A Deterministic Bottom-up Parser

The bottom-up parsing problem seems more difficult than the top-down problem. There are more choices that must be made. Not only must we somehow select a production, but we must decide on the part of the partially completed tree that it applies to. Nevertheless, we can often construct a systematic deterministic bottom-up parser with no backtracking.

Let us begin with a definition: the *skyline* of a sequence of bottom-up parsing trees (see figure 2.12 for an example) is the left-to-right sequence of their roots. The skyline will always be a right-most sentential form, provided that the input string is a sentence in the language. For example, in figure 2.12, the skyline is $E + (F * a)$, derivable from S by a right-most derivation.

Now we can introduce the bottom-up parsing machine, called an *LR(1) parser*. “LR” means “Left-to-right, Right-most”; the “1” refers to the

parser's need to examine at most one symbol in the input string past its current parsing point.

The parsing machine is shown in figure 2.17. It consists of a set of states (the circles) connected by transitions. Two kinds of transition appear: a *read* transition and a *lookahead* transition. The lookahead transitions are indicated by braces {...}. Note that the transitions are on the tokens in the terminal and nonterminal alphabets of the grammar G_3 .

The states 1 through 8 (marked with a #) are called *apply* states. In an apply state, the associated production can be attached to the right-most exposed tree skyline. The states 9 through 17 call for a read or a lookahead transition.

We apply this machine to some partial tree skyline, starting with the *start* state 9 and continuing until we hit an *apply* state. On each read transition, we match the current skyline token against a transition token, then move to the next token and state. On each lookahead transition, we match the current skyline token against a transition token, and move to the next state, but do not move on in the skyline. A lookahead transition may carry more than one token. For example, in state 9, token “.” or “d” is acceptable as a lookahead transition to state 4.

In an apply state, we have some production $A \rightarrow w$ indicated by the machine. The string w must then fit that part of the skyline we have just scanned. Notice that the machine “spells out” the string w upon falling into an apply state, associated with $A \rightarrow w$.

As in the top-down parsing machine, we do not need to draw a lot of trees to illustrate the process. We need only show the skyline string. The initial

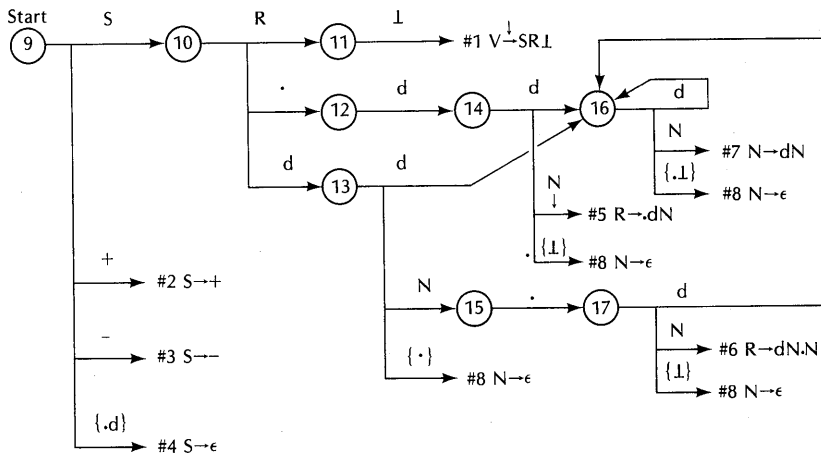


Figure 2.17. A bottom-up parsing machine for grammar G_3 .

skyline string is the input sentence, and the final, or halt, state is upon applying production # 1, $V \rightarrow SR \perp$ in the machine; the skyline will consist only of the start symbol V .

Let us trace this parsing machine with the input string $ddd.dd \perp$.

Skyline	State Path	Production
$ddd.dd \perp$	9	4, $S \rightarrow \epsilon$
$Sddd.dd \perp$	9, 10, 13, 16, 16	8, $N \rightarrow \epsilon$
$SdddN.dd \perp$	9, 10, 13, 16, 16	7, $N \rightarrow dN$
$SddN.dd \perp$	9, 10, 13, 16	7, $N \rightarrow dN$
$SdN.dd \perp$	9, 10, 13, 15, 17, 16, 16	8, $N \rightarrow \epsilon$
$SdN.ddN \perp$	9, 10, 13, 15, 17, 16, 16	7, $N \rightarrow dN$
$SdN.dN \perp$	9, 10, 13, 15, 17, 16	7, $N \rightarrow dN$
$SdN.N \perp$	9, 10, 13, 15, 17	6, $R \rightarrow dN.N$
$SR \perp$	9, 10, 11	1, $V \rightarrow SR \perp$
V	(halt)	

It should be clear that we have reproduced a right-most derivation of the sentence, in reverse order.

A syntax error is detected in this machine whenever we cannot find a transition from some state that matches the next input symbol. For example, if we reach state 12 and fail to see a “d” symbol, then there must be a syntax error at that point, and the input string cannot be in the language $L(G_3)$.

We shall further discuss bottom-up parsing in chapter 5.

Exercise

1. Trace the parser of figure 2.17 on the strings

$dd.d \perp$
 $.ddd \perp$
 $- .d \perp$
 $- . + d \perp$ {illegal}
 $d.d + \perp$ {illegal}
 $d.d. \perp$ {illegal}

2.4. Bibliographical notes

Some early papers on grammars and generating systems are found in Chomsky [1956]. A survey paper with additional references is Chomsky [1963]. The notation used for grammars and derivations is from Chomsky [1959]. References for most of the remaining material in this chapter may be found in the notes for the subsequent chapters.

CHAPTER 3

FINITE STATE MACHINES

A large digital system cannot be designed through a detailed electrical analysis of all its circuits. There are just too many components and the electrical circuit laws are too difficult to solve. The system as a whole can only be understood by a model that simplifies the system. One such model is the finite-state machine. In this model, a digital system is viewed as one that moves in discrete steps from one *state* to another. Each transition is determined by the state it currently is in, along with a set of inputs. In the transition, the machine may also output some discrete set of values.

A state in a digital hardware system is defined by some finite set of signal voltages, interpreted in a discrete manner (usually *high* or *low*). A state in a software system might be defined by the set of values of the storage registers, including the current position in the stored program.

The finite state machine model has many applications. Every digital computer system is conceptually a finite state machine, albeit one with a vast number of states. Many seemingly difficult language recognition problems yield to a finite state machine synthesis. Many computer subsystems, such as peripheral device controllers, tape formatters, etc., are first designed as finite state machines that are then transformed into their logic circuit equivalents.

We shall examine in detail only one class of finite state machines—the so-called *incompletely specified* no-output machines. These are particularly useful as language recognizers. We shall see that the class of finite state machines, or finite state automata, (FSA for short) is equivalent in recognition power to the class of regular grammars, and also to a special class of language generators called regular expressions. Many simple programming languages can be recognized by FSA.

An Example FSA

Before we formally define a finite-state machine, let us examine one that will serve as an example for the formal descriptions to come, figure 3.1. This machine recognizes a language consisting of the signed or unsigned decimal numbers.

An FSA consists of a set of states, transitions among the states, and an input string scanned by a read head. The read head starts at the left-most string token and moves to the right as the FSA moves from state to state.

The circles containing letters represent states. At any one time, the machine is in exactly one state. The state *S* is a *start state*. The machine is

placed in this state initially. The states B and H are called *halt* or *accepting* states, and are so indicated by the double circles.

The arrows connecting the states represent *state transitions*. Each one is labeled with a member of the alphabet of the language recognized by the machine. For this machine, the alphabet consists of four tokens:

$$\{d, +, -, .\}$$

where d represents one of the decimal digits $0, 1, 2, \dots, 9$.

As each token in the input string is scanned, the machine moves from state to state, according to the tokens on the arrows. For example, if the first token is “+”, then the first transition is from S to A. Then if the next token is a decimal point, “.”, the second transition is from A to G.

The FSA continues with its transitions until it either finishes the string, i.e., scans all the tokens, or it encounters a token that has no transition associated with it. If it scans the string and ends in a halt state (B or H), the string is said to be *accepted*. On the other hand, if it scans the string, but fails to end in a halt state, or if it is unable to scan the string because of a failure to find a matching transition on some token, then the machine fails to accept the string, and is said to *block*.

The FSA of figure 3.1 is *incompletely specified*, i.e., some states have no transitions on some tokens. For instance, state A has no transitions on “+” or “_”.

The FSA of figure 3.1 is designed to accept those strings in the form of a signed or unsigned decimal number and only those strings. For example, it will accept these strings:

```
- 15.
75.38
+.002
000001
+ 34.76
```

but will not accept these strings:

```
- 75 +
+ 17 - 56
3..14
.000.1
```

Now consider the specific string “+ 34.76”, which is accepted by the machine of figure 3.1. The transitions are

```
S to A on token “+”
A to B on token “3” (a digit d)
```

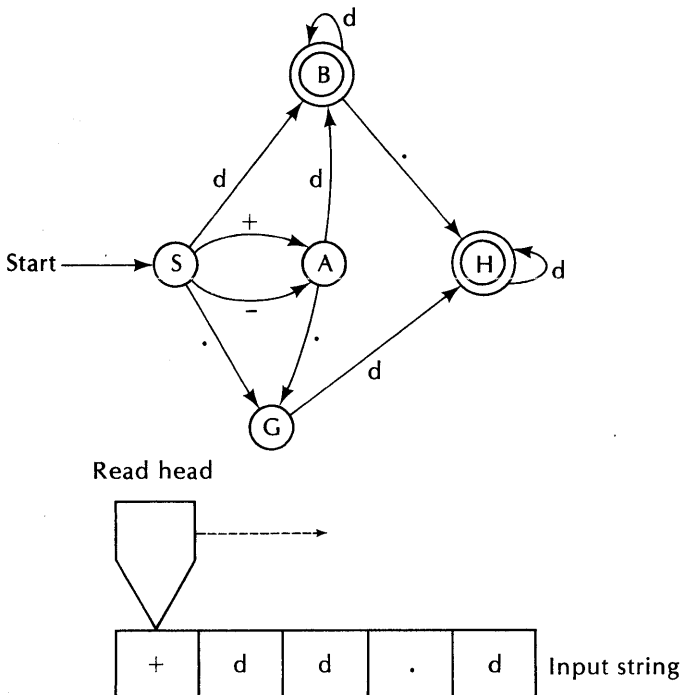


Figure 3.1. A finite-state automaton.

B to B on token "4"
 B to H on token "."
 H to H on token "7"
 H to H on token "6"

Since H is a halt state, and we have succeeded in scanning the entire string, the machine accepts "+ 34.76".

Now consider the string "+ 17-56", which will not be accepted by the machine. The transitions are

S to A on token "+"
 A to B on token "1"
 B to B on token "7"

At this point the scan must end, since there is no transition from state B on token "-". There is only one transition on "-", from state S; however, it is of no value now because the machine is not in state S. State B is a halt state, but the input list must also be completely scanned, and it has not been. Hence the machine fails to recognize "+ 17-56"; it blocks on the "-".

Finally, consider the string “+.” that will not be accepted. The transitions are

S to A on token “+”
A to G on token “.”

The FSA has scanned the entire string, but has ended in state G, which is not a halt state. Hence the machine has failed to recognize the string “+.”.

The value of such a machine in a computer system should be obvious—it provides a logically sound way to test input strings for membership in some language, that is, it serves as a *syntax checker*.

An FSA has many more applications. For example, we may associate an output string with each transition; we would then have a simple translator. We may also associate some general operation with each transition; such an FSA could then serve as a basis for a class of algorithms or as a machine controller, etc.

Exercises

1. Show that the FSA of figure 3.1 accepts these strings:

75.38
−15.
00000001
.000005

but not these:

+17−56
−75+
3. .14
.00.1

2. Suppose that only state H is a halt state in the machine of figure 3.1. Describe the language of the FSA informally.
3. Suppose that only state B is a halt state. Using the definition of acceptance above, what is the significance of an input string that leads to state H? Is state H of any value to the FSA as a language recognizer? Show informally that it may be removed. What other state can also be removed? What language is recognized by the resulting machine?

4. Extend the machine of figure 3.1 to accept decimal numbers with an exponent field, e.g.,

+3.7E+6

3.1. Formal definitions

We now provide a more formal definition of a FSA, one that will be useful in exploring its properties. A *deterministic finite-state automaton*, or DFSA, is a five-tuple $M = (Q, \Sigma, \delta, q_0, F)$, where

1. Q is a finite set of states.
2. Σ is a finite set of permissible input tokens, i.e., the alphabet of the machine.
3. δ is a partial function that maps a state and an input token to another state, called the *state transition function*.
4. q_0 is a designated state in Q , called the *initial* or *start state* of the FSA,
5. F is a subset of Q , consisting of at least one final state.

The FSA operates through a sequence of moves. Each move is dictated by the present state and the next input token to be scanned by the machine. The move consists of scanning the next input token, and simultaneously transferring from a “current” state to a “next” state (which may be the same as the current state). A move may be made only if the δ function permits it to be made; the current state and the next token must map to another state through the δ function.

For example, consider the FSA of figure 3.1. Its state set $Q = \{S, A, B, G, H\}$, q_0 is S , its halt set $F = \{B, H\}$, its alphabet $\Sigma = \{+, -, d, .\}$, and its mapping function is:

$$\begin{aligned}\delta(S, +) &= A \\ \delta(S, -) &= A \\ \delta(S, .) &= G \\ \delta(S, d) &= B \\ \delta(A, d) &= B \\ \delta(A, .) &= G \\ \delta(B, d) &= B \\ \delta(B, .) &= H \\ \delta(G, d) &= H \\ \delta(H, d) &= H\end{aligned}$$

For example, the function δ maps state A and token “.” to G. This corresponds to the transition A to G under “.” in figure 3.1. The state transition function does not map all possible states and tokens to states; those state-token pairs that are not mapped are not permitted as automaton moves.

Transition Function as a Table

The transition function for a FSA may be expressed as a table. The input tokens are listed along the top and the states along the left side (figure 3.2). The table contains the mapping $\delta(P, a)$, where P (a state) defines a row and “a” (a token) defines a column. A blank entry means that δ is undefined for that state and input.

Configurations

Suppose that a FSA has completed a number of moves in a string. To predict its future behavior, we need only know the remainder of the input string, starting with the next token, and the current state. These two items of information provide a complete description of the FSA at a particular point in a particular application, and will be called a *configuration*. A configuration will be designated (q, w) , where q is a state and w is the string remaining to be scanned.

The configuration (q_0, w) , where q_0 is the start state and w is any string to be accepted or rejected by the automaton is called an *initial configuration*. A configuration (q, ϵ) , where ϵ is the empty string, is called a *final configuration*, provided that q is in F , the set of halt states.

A *move* of the machine (designated “ $\vdash$ ”) connects one configuration to another. We have

$(q, aw) \vdash (q', w)$ if and only if “a” is in Σ , w is in Σ^* , and $q' = \delta(q, a)$.

which means that given a machine in state q , with the input string “aw” (“a” is the first token and “w” is the rest of the string), one move results in state q' and string “w”. The token “a” has been scanned by the move, leaving the rest of the string “w”. The move is only possible if the state transition function δ yields a state q' for the current state q and input token “a”.

A sequence of moves of the machine may be designated $\vdash^*$ or $\vdash^+$. The $+$ means “one or more moves,” and the $*$ means “zero or more moves”. A zero move results in no change in state and no scan of the input string. The sequence $\vdash^+$ is called the *transitive closure* of $\vdash$, and $\vdash^*$ is called the *reflexive transitive closure* of $\vdash$.

With this notation, we may succinctly define the language $L(M)$ recognized by a FSA M :

$$L(M) = \{w \in \Sigma^* \mid (q_0, w) \vdash^* (q, \epsilon) \text{ for some } q \text{ in } F\}$$

		Input token				
		δ	+	-	.	d
States	S		A	A	G	B
	A				G	B
	$\textcircled{B}$				H	B
	G					H
	$\textcircled{H}$					H

Figure 3.2. Finite-state automaton of figure 3.1 as a table, expressing the transition function δ .

which means that the language $L(M)$ is the set of strings w such that the FSA can begin in start state q_0 , scan through string w , and end in a halt state when the string is completely scanned.

Exercise

Describe the acceptance of the string $+.002$ by the FSA of figure 3.1 as a sequence of configurations, starting with $(S, +.002)$ and ending with (q, ϵ) , where q is in $\{B, H\}$.

Machine Equivalence

Two machines, M and M' , are said to be syntactically equivalent if they recognize the same language, i.e., if $L(M) = L(M')$. The machines need not have the same number of states, nor must the states carry the same state labels. This definition is equivalent to the statement:

M and M' are equivalent if and only if for every string x , M accepts x if and only if M' accepts x .

If a machine M can be transformed into a machine M' by merely relabeling its states, then M and M' are said to be *isomorphic*.

A fundamental theorem of FSA is that for every machine M , there exists an equivalent machine M' with a minimal number of states, and that every machine M'' with the same number of states as M' , and equivalent to M' , must be isomorphic to M' , i.e., M' is structurally unique.

We shall expand upon this notion of equivalence later, and show how an arbitrary machine can be reduced to minimal form.

Nondeterministic Finite-State Automata

An FSA is said to be *deterministic* when no choices are provided in any of its moves. Every move is absolutely determined by the current state and the next token, clearly a desirable machine for any implementation. A *nondeterministic* FSA is such that some arbitrary choices are permitted in some of its transitions. There are some states and input tokens for which more than one transition may be taken. A number of concepts are easier to express nondeterministically than deterministically. We shall also show that, given a nondeterministic automaton, we can always systematically convert it into a deterministic automaton that recognizes the same language.

A *nondeterministic FSA*, or *NDFSA* for short, is defined exactly as a deterministic FSA, with two exceptions:

1. Some moves may involve a choice. This choice is represented by a state transition function δ that maps a state-next-token pair into a set of states. The set may consist of a single state, in which case no choice is provided for that particular state-next-token pair. However, in general some of the state-next-token pairs map to two or more states. We therefore use the notation:

$$\delta(q, a) = \{\text{some set of states}\}$$

for a nondeterministic transition function.

2. Some moves may be made without scanning the next token. Such a move is called an *empty move*, and may be included in the state transition function by the notation

$$\delta(q, \epsilon) = \{\text{some set of states}\}$$

There may be one or more possible next states. An empty move may be invoked (if it exists) even when the string has been completely scanned. In this way, it may be possible to reach a halt state through one or more empty moves from a non-halt state.

We say that a nondeterministic finite automaton accepts a string w if there exists some sequence of moves, beginning with the start state and ending in a halt state that scans the entire string. It is not necessary that each sequence of choices leads to acceptance; only one sequence is necessary.

Figure 3.3 gives an example of a nondeterministic FSA. Its transition table is figure 3.4. This machine happens to recognize the same language as the FSA of figure 3.1. We shall prove this in due time. Note that it contains a number of empty moves, S to A , A to E , etc. Also note that in state A , there are three possible moves on token “d”. The machine may scan the “d” and transfer to either B or C , or it may make an empty move to E and then scan “d”.

Now consider the recognition of the string “-24.57” by the NDFSA of figure 3.3. A correct choice would be the “-” transition from the start state S to A ; the machine might also choose the empty transition to A . However, the

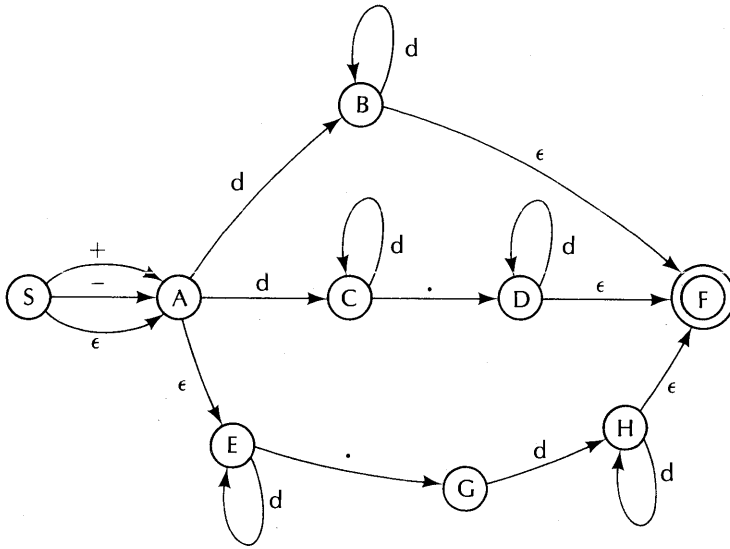


Figure 3.3. A non-deterministic machine equivalent to the machine in figure 3.1.

empty move results in a block, since there is no way to scan “-” once the machine reaches state A.

When in A, a choice among three possible paths exists, to state B, C, or E. Depending on the next token, any of the three may be possible moves. However, a move along the upper path, through B, means that a decimal point can never be scanned. An examination of the remaining two paths reveals that either one is satisfactory for our example, and results in acceptance of the string “-24.57”. Thus, the middle path yields the state sequence A, C, C, D, D, D, F. Note again the nondeterminism of the final transition D to F. The machine is in D when the last token (a digit) is read, but may continue to make more empty moves to reach a final state (F).

Although it appears that a nondeterministic machine is more “loose” in its recognition capability, this is not really the case. We challenge the reader to find a string recognized by the automaton of figure 3.3 that is not recognized by the automaton of figure 3.1. The two automata are equivalent.

Exercises

1. Find a sequence of accepting moves for each of the following strings in the NDFSA of figure 3.3:

+ .004

56.3

-334

δ	Input Symbols				
	+	-	.	d	ϵ
S	A	A			A
A				B,C	E
B				B	F
C			D	C	
D				D	F
E			G	E	
F					
G				H	
H				H	F

Figure 3.4. Tabular form of the non-deterministic finite automaton of figure 3.3.

2. Show that each of the following strings cannot be accepted by the NDFSA of figure 3.3, by exploring all the possible move sequences:

+0.0.
-2.+3
..02

3. Consider the NDFSA of figure 3.3, but with one of the three paths starting with A removed (there are three such machines). Discuss the three languages informally.
4. Design a simple NDFSA such that the acceptance of a string of finite length can be made in an indefinitely large number of moves.

A Backtracking Machine Model for a NDFSA

A nondeterministic FSA can be modeled by a backtracking system of the sort described in section 2.3.2. We do not propose implementing a FSA in this fashion; we merely present the model as another means of viewing a nondeterministic automaton.

Recall that a backtracking problem-solving system has three kinds of moves: a *forward* move, a *backtracking* move and a *choice* move. For a NDFSA, we may make each state transition a choice move, for the sake of generality, whether a state in fact has any choices or not. The backtracking move is invoked on any failure to accept the string. The forward move is simply a transition to the next state, scanning a string character in the process. The input string will be scanned left-to-right by a read head, however, the read head is permitted to move backward in a backtracking move.

The backtracking system tape T will have cells containing the state number, the position of the input token on the input list, and the particular state transition adopted, figure 3.5.

Initially, the tape T is empty and positioned at its left-most end; the machine M is in the start state; and the read head is positioned at the left-most token of the input list.

Upon leaving any state Q in a forward move, a cell on tape T is written.

If M blocks, the tape T is backed up cell by cell, until a cell is found such that an alternative move is found. The read head of M is then set to the position indicated by the cell, the state of M is set, the alternative move is made, and the cell is replaced by a new cell characteristic of the new move.

This process is repeated until one of two things happens:

1. The tape T is backed up to the first cell, and this cell indicates that no alternative moves exist. In this case, the input string is not in the machine's language.
2. The machine M ultimately reaches a halt state, and the input string has been completely scanned. In this case, the input string has been accepted by M. Tape T contains a record of the moves.

Now consider the NDFSA in figure 3.5, and let the transitions in each state be ordered from top to bottom in a clockwise sense. Thus transition "d" from A to B is labeled 1, transition d from A to C is labeled 2, and the empty transition from A to E is labeled 3. We also need string positions; let these be 1, 2, and 3, respectively:

—	3	.
1	2	3

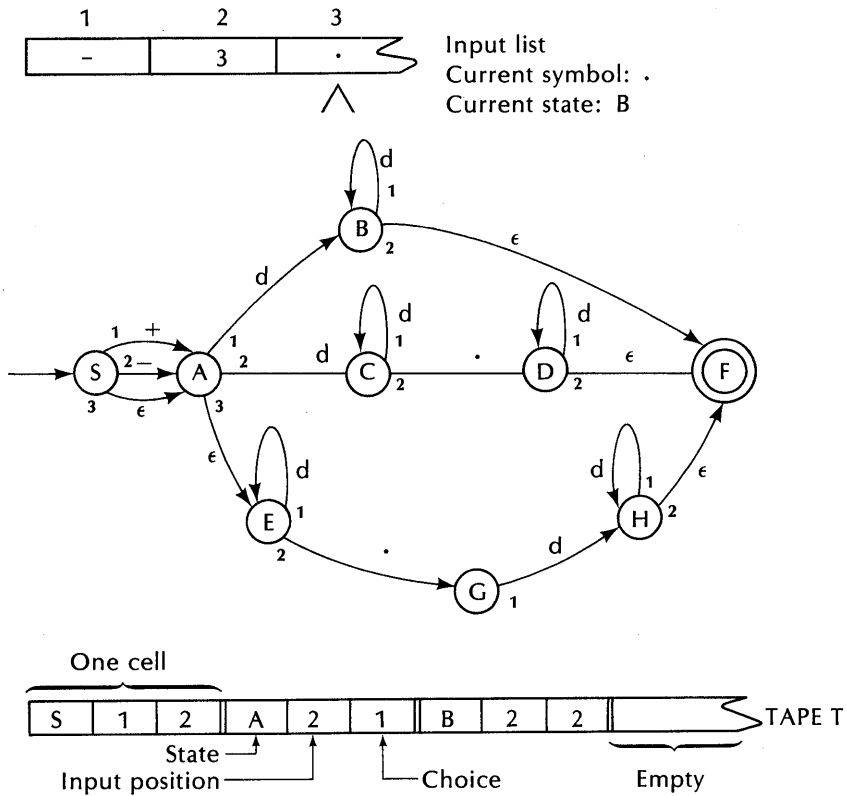


Figure 3.5. Backtracking machine M and its tape T, shown in one configuration. Input string: “-3.”.

Let us trace some of the moves of the system for the above string. Initially, T is empty.

The first move from state S must be on the minus transition, 2, (first choice) so the first cell reads (S, 1, 2). The next move involves a digit “3” in position 2, from state A. We have three possible moves: to state B, to state C, or to state E. We take the first one, creating the cell (A, 2, 1). Tape T now reads (S, 1, 2) (A, 2, 1). The next move, from state B, calls for a decimal point. Although there is no such transition from B, the empty move to F may be taken. This yields the tape (S, 1, 2) (A, 2, 1) (B, 2, 2). Note that the input string position is unchanged. Figure 3.5 shows a snapshot of machine M, its input string, and the tape T at this point in the process.

We now find machine M in state F, with no more possible moves, and the input list incompletely read—which is not an acceptance condition, hence we must back up tape T and examine other alternatives. The last cell, (B,2,2), offers no hope, since there are no other moves out of B on the last token “.”.

The next-to-last cell, (A,2,1), does provide another alternative. The cell indicates that the machine M chose the A to B transition on token “3”. We can also move to C on this token, hence we do so, yielding the modified tape: (S, 1, 2) (A, 2, 2).

We now find the machine M in state C, with token “.”. Clearly, the move from C to D (number 2 by the ordering scheme) is legal and yields the tape (S, 1, 2) (A, 2, 2) (C, 3, 2), with machine M in state D. The input list is now fully scanned; however, the empty move to F is legal, yielding acceptance and the final tape $T = (S, 1, 2) (A, 2, 2) (C, 3, 2) (D, 3, 2)$.

As we shall see, it is never necessary to implement a nondeterministic finite automaton with a backtracking tape, because a nondeterministic finite automaton can always be transformed into an equivalent deterministic finite automaton. A deterministic automaton never needs to back up. If it blocks on a token, there exist no choices that have been made arbitrarily in its previous moves, and the block is therefore sufficient proof that the string is not in the machine’s language.

Exercises

1. Trace the acceptance of the following strings through the backtracking system, figure 3.5:

— .16
3.6
9.

2. Show that the following strings fail to be accepted by tracing the backtracking system:

— .
.3.

3. Show that the backtracking system will fail if an empty move cycle exists in the machine M, through an example. An *empty move cycle* is a sequence of transitions from some state A back to A, all with empty moves. Why will the system fail?
4. Show that the system will always terminate on a finite input string if no empty move cycle exists in the FSA.

3.2. Transformation of a NDFSA to a DFSA

The transformation of a NDFSA to a DFSA is accomplished by the following steps: (1) detection and removal of empty move cycles, (2) removal

of the remaining empty moves, and (3) transformation into a deterministic FSA.

3.2.1. Empty Cycle Detection and Removal

An *empty move cycle* is a sequence of empty transitions that begins with some state A and ends in state A. All the states in such a cycle are clearly equivalent, since we may get from any one of them to any other on any input token, without changing the read-head position.

An empty move cycle may be eliminated by merging their states. A set of states is *merged* by giving them all a common name. This has the effect of causing a transition into or out of any one of the empty cycle states to effectively be a transition associated with all of the states. If any one of the merged states is a halt state, the newly named state must also be a halt state.

Example

Figure 3.6 shows a machine with several empty cycles, ACD, etc. The ACD empty cycle may be collapsed by merging states A, C, and D. This merger yields figure 3.7, in which the ACED empty cycle has become the cycle AE. Collapsing this one yields the machine of figure 3.8, which contains no empty cycles.

Empty move cycles are detected and removed by the following algorithm.

Algorithm 3.1. Empty cycle removal

Let each state carry a mark in the set $\{0,1\}$. Mark 1 indicates that the state has been *considered*. Initially, every state carries mark 0 (*not considered*).

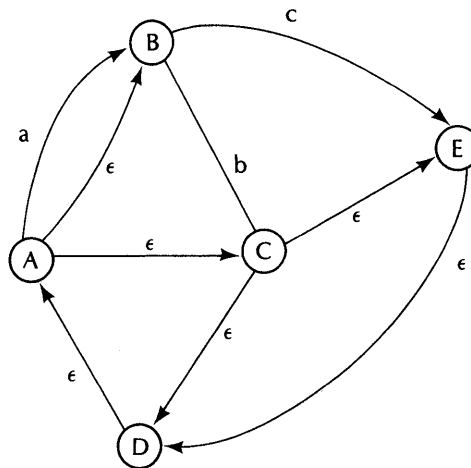


Figure 3.6. A finite-state automaton with several empty cycles.

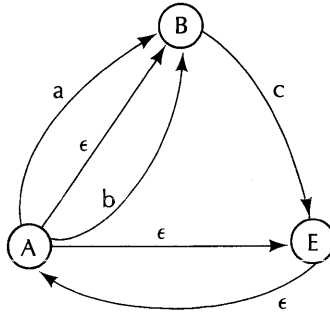


Figure 3.7. The finite-state automaton of figure 3.6 with the ACD empty cycle removed by merging states A, C, and D.

1. Choose any state p with mark 0. We then construct a tree whose nodes are states. Its root is p , and the children of any state q are those states for which an empty move from q exists. The construction of any path is terminated on a node with no empty moves, or on a node q' such that q' appears anywhere else in the tree (whether on that path or not). The tree is obviously finite, since it can contain at most as many nodes as there are states.

2. If node p appears twice in the tree, once as the root and again on some node N , then the path from the root to N represents the states on an empty cycle. Merge these states, and return to step 1. (Note that p remains unmarked).

3. If node p appears exactly once in the tree, as the root, then there are no empty cycles containing p . Mark p (1) and go to step 1.

The number of steps in this algorithm is clearly finite, and it is easy to show that at its conclusion, the machine M contains no empty cycles.

Exercise

Construct such a tree for the FSA of figure 3.6 and for each of the states B and C.

3.2.2. Removal of Empty Transitions

Once all the empty cycles have been eliminated, the remaining noncyclic

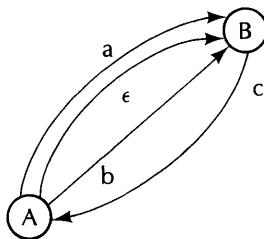


Figure 3.8. The finite-state automaton of figure 3.7 with all empty cycles removed by merging states.

empty moves may be removed. Consider some state p in machine M , with empty moves to states $q_1, q_2, q_3, \dots$ (figure 3.9). This part of machine M is expressed by the transition function

$$\delta(p, \epsilon) = \{q_1, q_2, \dots\}$$

Now each of the states $q_1, q_2, \dots$ have transitions (in general) to other states $r_1, r_2, \dots$ on tokens $a_1, a_2, \dots$. Some of the r states may be q states or the p state, and some of the tokens $a_1, a_2, \dots$ may be empty. However, no transition from p through q to itself can consist only of empty transitions.

Clearly, the empty move from p to q_1 can be eliminated if we add to p 's moves the moves:

p to r_1 on a_1 , and

p to r_2 on a_2 .

The idea is that if M can reach r_1 from p on an empty move and then on a_1 , then an equivalent move is from p to r_1 on a_1 directly.

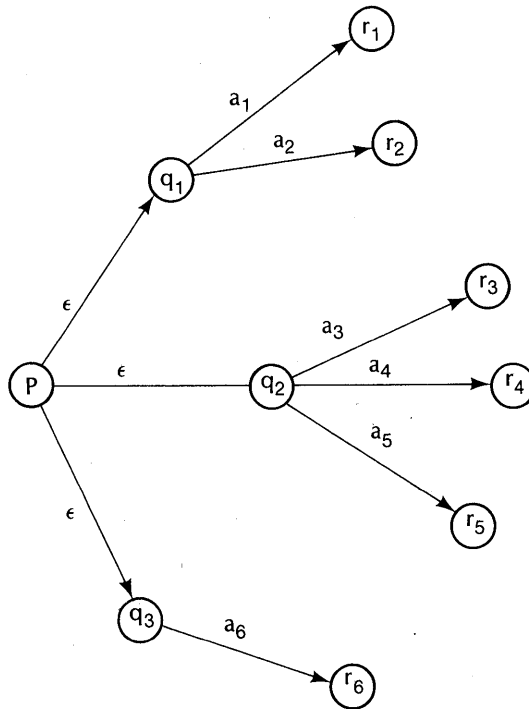


Figure 3.9. A state p with empty moves to states $q_1, q_2, \dots$

Similarly, the empty move from p to q_2 may be eliminated by adding the moves

p to r_3 on a_3 ,

p to r_4 on a_4 , and

p to r_5 on a_5 .

One more operation must be performed: If q is a halt state and an empty move from p to q exists, then p must be added to the set of halt states upon removing the empty move. We observe that if the input string is completely scanned when M is in state p , then M may move to q on no token and accept the string. If the empty move to q is removed, then this would no longer be possible; hence p must become a halt state.

The algorithm for this process follows.

Algorithm 3.2. Removal of empty transitions

Given an empty transition p to q on ϵ ; i.e., q is a member of $\delta(p, \epsilon)$. Set $\delta(p, \epsilon) = \emptyset$, and add r to $\delta(p, a)$, for every a and r such that r is in $\delta(q, a)$. If q is in F , then p must be added to F .

This algorithm may result in one or more new empty transitions from p to some state r , and will therefore have to be repeated. However, it must ultimately end with no empty transitions from p , given that machine M contains no empty cycles. The argument is essentially that n repetitions of this algorithm involve state p , and a sequence of states r_1, r_2 , etc., each of which must be distinct. If the r 's were not distinct and different from p , then there must exist an empty cycle, a contradiction.

For example, consider figure 3.3. Although there are no empty cycles, there are five empty moves. Consider the empty move from H to F . There are no moves from F in this machine. However, since F is a halt state, H must become a halt state. Similarly, the empty moves from B to F and from D to F may be removed by making B and D halt states. We end up with a machine with four halt states B, D, H , and F . Since F can no longer be reached from the start state, state F is called an *inaccessible* state. There is no point in keeping inaccessible states, so machine M looks like figure 3.10 after removing these three empty moves and state F .

Next consider the empty move from S to A in figure 3.10. The rule is that we replace it with three new transitions from S to B, C , and E . Since A is not in the halt set, S is not added to the halt set. The result is the machine of figure 3.11, which still has two empty moves. One of them came from the empty move A to E . (Trust us—the reduction process is not caught in an infinite loop.)

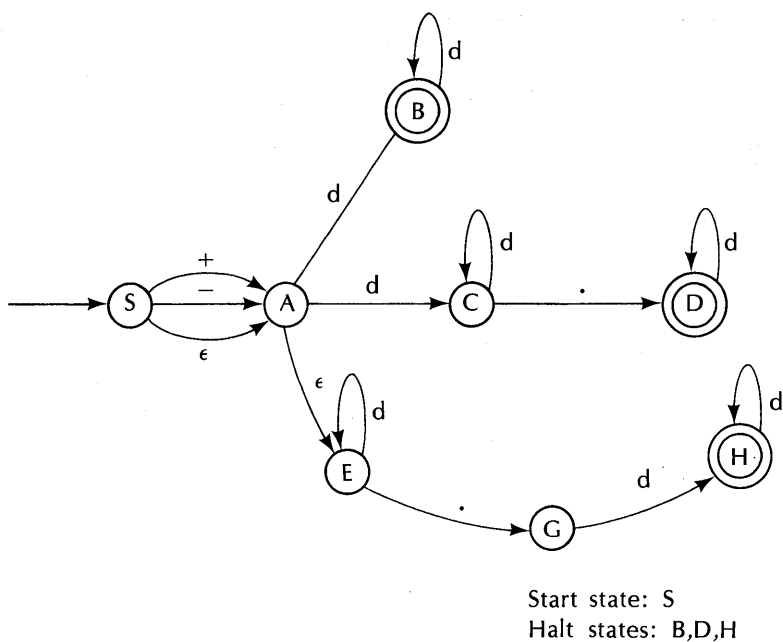


Figure 3.10. The finite-state automaton of figure 3.3 with the empty moves to F removed.

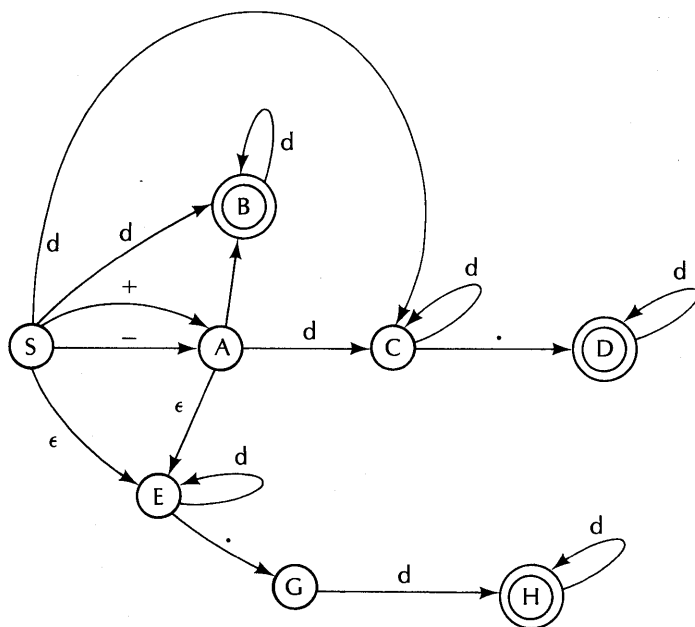


Figure 3.11. The finite-state automaton of figure 3.10 with the S to A empty moves removed.

Consider next the empty move from A to E. Its removal means that state A picks up transitions to E on “d” and to G on “.”. Finally, the removal of the empty move from S to E means that state S picks up transitions to E on “d” and to G on “.”. The final machine M' , free of empty moves, is shown in figure 3.12.

Removal of Empty Moves Using a Transition Table

A more systematic way of empty move reduction is through use of the table representation for the FSA. We first identify every state with an empty move to a halt state. When one is found, it is marked as a halt state. Considering figure 3.4, states B, D, and H have empty moves to F. Hence they can be circled, yielding figure 3.13. There are no other states that need be marked as halt states, since there are no empty moves to B, D, or H.

Now consider the empty move from state A to state E, figure 3.13. For token “+”, $\delta(E, +)$ has no members. Hence $\delta(A, +)$ remains empty. Similarly for token “-”. For token “.”, $\delta(E, .)$ contains state G. Hence, we add state G to $\delta(A, .)$. In the same manner, for token “d”, state E is added to $\delta(A, d)$. Figure 3.14 shows the resulting “A” row.

This operation is continued for every state with an outgoing empty transition, until no further additions to the table can be made. When this point is reached, all the empty moves may be dropped by crossing out the empty move column. Thus in figure 3.13, nothing is added to the table by the

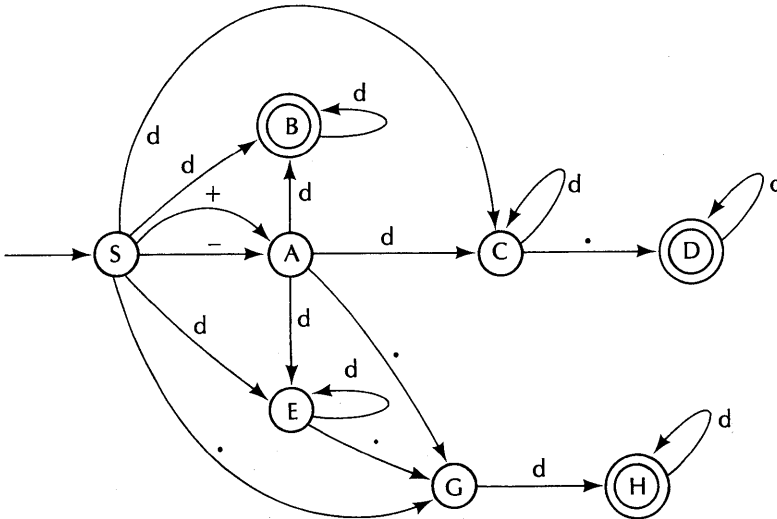


Figure 3.12. The finite-state automaton of figure 3.11 with all empty moves removed.

		Input symbols				
δ		+	-	.	d	ϵ
States	S	A	A			A
	A				B,C	E
	(B)				B	F
	C			D	C	
	(D)				D	F
	E			G	E	
	(F)					
	G				H	
	(H)				H	F

Figure 3.13. Tabular form of the non-deterministic finite-state automaton of figure 3.5, with the halt states marked.

empty moves from B, D, and H to F, since F is empty anyway. However, the S row is expanded by the A row's states because of the empty transition from S to A. The final table, representing a FSA free of empty moves, is given in figure 3.14.

3.2.3. Transformation from Nondeterministic to Deterministic

The machine of figure 3.12 (or figure 3.14) is still nondeterministic. For example, there are three transitions from state A on a "d". Removal of the empty moves has not changed this situation; indeed, it has aggravated it.

The remaining nondeterministic moves of an NDFSA with no empty moves stems from one or more states with several moves on the same token possible. There are two such states in figure 3.12, S and A. We can resolve the choices in these states by calling each set of states a new state; the new state then will be the merger of its component states. Thus we create a new state "BCE" which will receive the merger of the transitions from states B, C, and E. We are in a sense deferring the choice on token "d" and state S by introducing a new target state "BCE".

The general method is defined in algorithm 3.3, as follows:

Algorithm 3.3. Converting a NDFSA M into an equivalent DFSA M'

1. The states of M' consist of sets of states of M . That is, if A , C , and F are states in M , then $\{A\}$, $\{C\}$, $\{F\}$, $\{A, C\}$, $\{C, F\}$, $\{A, F\}$, $\{A, C, F\}$ are states in M' . Since it is unusual to think of a set of states as a state, we change our notation slightly by using brackets $[]$ instead of braces $\{\}$ to represent a state in M' . Then $[A, C]$ is the name of a state in M' , where A and C are states in M .

Although there are many possible sets of states of M , the maximum number is finite; indeed, if there are n states in M , then the largest possible number of states in M' is $(2^n - 1)$. This can obviously be a very large number. Fortunately, most of the states in M' are inaccessible and need never appear in the reduction process.

2. If P is a halt state in M , then every state $[\dots, P, \dots]$ containing P in M' is a halt state in M' .

3. If S is the start state in M , then $[S]$ is the start state in M' .

4. Let $[P_1, P_2, \dots, P_n]$ be a state in M' . Then consider all the transition functions

		Input symbols				
δ		+	-	.	d	ϵ
States	S	A	A	G	B,C,E	A
	A			G	B,C,E	E
	(B)				B	F
	C			D	C	
	(D)				D	F
	E			G	E	
	(F)					
	G				H	
	(H)				H	F

Figure 3.14. Empty move removal. The ϵ column may be deleted.

$$\delta(P_1, a), \delta(P_2, a), \dots, \delta(P_n, a)$$

on some token “a” in M . We then construct a new transition function δ' in M' as follows:

(a) Let $\delta(P_1, a) \cup \delta(P_2, a) \cup \dots \cup \delta(P_n, a) = \{Q_1, Q_2, \dots, Q_r\}$. That is, we collect all of the states to which the states $P_1, P_2, \dots, P_n$ transfer on token “a”, and call these $Q_1, Q_2, \dots, Q_r$.

(b) Then set $\delta'([P_1, P_2, \dots, P_n], a) = [Q_1, Q_2, \dots, Q_r]$. Note that this is a deterministic transition function, since the M' state $[P_1, P_2, \dots, P_n]$ on token “a” transfers to exactly one state $[Q_1, Q_2, \dots, Q_r]$.

5. Step (4) is repeated for every state in M' and every transition token “a”.

This algorithm looks formidable, but in fact it is quite easy, particularly when carried out on a state table, as we shall see.

Tabular Reduction of a NDFSA to a DFSA

Consider figure 3.14. Its FSA may be transformed into a DFSA through row operations similar to those used for removal of the empty moves. The nondeterminism of figure 3.14 is accounted for by a multiple state set in the S and A rows, in the “d” column. According to algorithm 3.4, we need a new state $[B, C, E]$ in M' , since the set $\{B, C, E\}$ appears in a transition in M . Let us therefore add such a state to the table. The transitions from the new state $[B, C, E]$ consist:

- For token “+”, of NULL, since neither of the states B, C , or E has a transition on token “+”.
- For token “-”, of NULL.
- For token “.”, of a transition to state $[D, G]$, since D is reached from C on token “.”, and G is reached from E on token “.”.
- For token “d”, of a transition to state $[B, C, E]$, since B goes to B on “d”, C goes to C on “d”, and E goes to E on “d”.

Thus the new row for state $[B, C, E]$ appears as shown in figure 3.15. Since the new state $[B, C, E]$ contains a halt state (B) in M , it is marked as a halt state in M' .

In the process, we have introduced another state, $[D, G]$. This is a halt state. Its development in the table leads to the definition of another halt state, $[D, H]$. The algorithm ends on state $[D, H]$, since every state appearing in the table is defined as some row in the table. The final DFSA is in figure 3.16.

Algorithm 3.3 generates a machine M' from a machine M , such that M and M' are equivalent. A proof of this assertion will be given later in the chapter, after we have defined equivalence more formally.

		Input symbols			
δ		+	-	.	d
States	S	A	A	G	B,C,E
	A			G	B,C,E
	(B)				B
	C			D	C
	(D)				D
	E			G	E
	(F)				
	G				H
	(H)				H
	(B,C,E)			D,G	B,C,E
					} New states

Figure 3.15. New composite state {B,C,E} created.

3.2.4. Accessible States

Some of the states in figure 3.16 cannot be reached from the start state, for example, state B. We may therefore delete all the inaccessible states from the DFSA, by using the following algorithm, which should be self-evident:

Algorithm 3.4 Detection of accessible states in a DFSA

1. Mark the start state S.
2. Given any marked state P, mark every state Q such that a transition from P to Q on some input token exists.
3. Repeat step (2) until no more states can be marked.

Upon completing algorithm 3.4, every nonmarked state is inaccessible from the start state, and may therefore be discarded.

Algorithm 3.4 applied to figure 3.16 shows that states B, C, D, E, and F are inaccessible. What happened to them? State F served only one purpose in the machine of figure 3.13—that of providing a halt state. But we have marked

		Input symbols			
δ		+	-	.	d
States	S	A	A	G	{B,C,E}
	A			G	{B,C,E}
	(B)				B
	C			D	C
	(D)				D
	E			G	E
	(F)				
	G				H
	(H)				H
	(B,C,E)			{D,G}	{B,C,E}
	(D,G)				{D,H}
	(D,H)				{D,H}

New states

Figure 3.16. Completion of new state creation.

several accessible states in figure 3.16 as halt states, and they got that way through empty transitions to F. Hence F “lives on” in its presence in some other states.

State B, to examine another one, survives in the composite state [B, C, E],

similarly, C and E. State D survives in the composite states [D, G] and [D, H]. So while they are gone, they have left their mark on the FSA.

The DFSA, with its inaccessible states removed, is shown in figure 3.17.

Exercise

1. Reduce the following NDFSA to a DFSA, and remove the inaccessible states:

δ	input token				
	+	()	*	.	ϵ
S	A		C		
A	D		D		B
B	A	B		C	E
C	D		F		A
D		E		B	
E	C		F	D	F
F*					B

(the * indicates a halt state).

δ	Input symbols			
	+	-	.	d
S	A	A	G	B,C,E
A			G	B,C,E
G				H
States (H)				H
(B,C,E)			D,G	B,C,E
(D,G)				D,H
(D,H)				D,H

Figure 3.17. Inaccessible states removed.

3.3. Machine Equivalence

We have until now used the notion of machine equivalence without much development. Two machines M and M' are equivalent if they accept the same language. That definition has sufficed thus far. We now develop the concept of equivalence more formally, and will arrive at an algorithm for reducing the number of states in a finite-state automaton to the least possible. The reduction method will also enable us to decide whether two seemingly different finite-state machines are in fact equivalent. The general notion of language acceptance is not practical, since we can seldom try all possible strings on both machines and test their acceptances.

The reduction of a machine to the fewest possible states is obviously of economic value. Among other things, it will reduce the task of designing semantic actions for the machine to a minimum.

3.3.1. Definitions

We start by defining the *k-equivalence* between two states P in M and P' in M' , where k is some integer, $k \geq 0$ and M and M' may be the same machine.

State P in M and state P' in M' are said to be *k-equivalent* if, for every string x of length k or less, machine M in state P accepts x if and only if machine M' in state P' accepts x .

If states P and P' are not k -equivalent, then they are said to be *k-distinguishable*; there is then some string x of length k or less, such that either: (1) machine M in state P accepts x , but machine M' in state P' does not, or (2) machine M' in state P' accepts x , but machine M in state P does not.

Two states P and P' are said to be *equivalent* if they are k -equivalent for all k .

A machine M is said to be *reduced* if no state in its state set is inaccessible and no two distinct states are equivalent.

A pair of equivalent states P and Q in a machine M may be merged by changing the name “ Q ” to “ P ” everywhere, without affecting the language recognized by the machine.

Let $P \stackrel{k}{\equiv} Q$ denote k -equivalence of states P and Q in a FSA M . Obviously $P \stackrel{k}{\equiv} P$, and by the symmetry of the definition, if $P \stackrel{k}{\equiv} Q$, then $Q \stackrel{k}{\equiv} P$. The k -equivalence is an example of a relation, and any relation that satisfies these two properties is said to be *symmetric* and *reflexive*.

The k -equivalence relation is also *transitive*: If $P \stackrel{k}{\equiv} Q$ and $Q \stackrel{k}{\equiv} R$, then $P \stackrel{k}{\equiv} R$. It is easy to show that k -equivalence is transitive. Let x be any string of length k or less that is accepted in state P . Then if $P \stackrel{k}{\equiv} Q$, it is also accepted in state Q . If $Q \stackrel{k}{\equiv} R$, it is also accepted by state R . Hence $P \stackrel{k}{\equiv} R$.

Any relation that is symmetric, reflexive, and transitive is called an *equivalence relation*. A fundamental property of an equivalence relation, and

one that we shall exploit in reducing a FSA to its minimal form, is the following:

An equivalence relation R upon a finite set of objects S partitions S into disjoint subsets, such that any two members of any subset are equivalent to each other, and no two members of different subsets are equivalent to each other.

This assertion may be proven by first considering how a set S is divided into subsets by an equivalence relation. Let the members of S be $a_1, a_2, \dots, a_n$. Then we create a sequence of subsets $S_1, S_2, \dots$ of S as follows. Each of $S_1, S_2, \dots$ is initially empty. S_1 is created by placing a_1 into it, then including a copy of every other member of S that is equivalent to a_1 . Note that by symmetry and transitivity, these must be equivalent to each other. When S_1 is completed, there may or may not be some members of $S - S_1$ left over. Suppose there are some members of $S - S_1$; call these $b_1, b_2, \dots, b_m$. We then start a new subset S_2 by placing b_1 in it, then adding all the members of S that are equivalent to b_1 . The interesting question is whether there can be a state that belongs to both S_1 and S_2 . Suppose there were; let it be called Q . Then by transitivity and reflexivity, Q must be equivalent to a_1 , because it is in S_1 , and also to b_1 , because it is in S_2 . But then by transitivity a_1 is equivalent to b_1 . We are led to a contradiction, since b_1 was specifically one of the states left out of set S_1 when we first collected together the states equivalent to a_1 . Hence set S_1 and S_2 must be disjoint. A similar argument applies to sets S_3, S_4 , etc.

Now suppose that we have somehow partitioned the state set of a FSA by k -equivalence. What is the nature of the partition induced by $(k+1)$ -equivalence? The answer is that a $(k+1)$ -equivalence is a *refinement* of a k -equivalence. By refinement, we mean either that the partition of a $(k+1)$ -equivalence is exactly the same as that of a k -equivalence or that some of the subsets in the k -equivalence have become further subdivided. The boundaries between the subsets of a partition are not changed by a refinement; rather, additional boundaries are introduced.

Refinement may be illustrated as follows. Suppose we have a set S of states $A, B, \dots, J$, in a machine M , and they are partitioned as follows:

$$S = \{A, D, I\} \{B, C\} \{E, F, G, H\} \{J\}$$

Note that each state appears exactly once and belongs to exactly one subset. A refinement of this partition might be the following example:

$$\{A\} \{D, I\} \{B, C\} \{E, F\} \{G, H\} \{J\}$$

When at least one subset of a partition is subdivided in a refinement, the refinement is called *proper*. The above example is a proper refinement.

The following partition is NOT a refinement of the partition S :

$$\{A, B, D\} \{I, C\} \{E, J\} \{G, H\} \{F\}$$

Although there are more subsets, states I and C have become members of a common subset, whereas in S they were in disjoint subsets.

We now prove our assertion: that a $(k+1)$ -equivalence induces a refinement on the partition induced by a k -equivalence. We need only show that a pair of states P and Q that were disjoint in the k -equivalence partition S remain disjoint in the $(k+1)$ -equivalence partition S' .

To prove this assertion, recall that P and Q are disjoint in S because there exists some string x of length k or less that distinguishes these two states. But this string is also of length $(k+1)$ or less, consequently P and Q must be disjoint in S' .

3.3.2. Reduction

We are at last in a position to reduce a FSA to its minimal form in a systematic manner. We need only these observations, which should be evident from the preceding discussion:

1. The 0-equivalent partition of the state set of a machine M is $\{F\}$, $\{Q-F\}$. That is, the 0-equivalent partition consists of the halt states and the non-halt states. These must be in separate partitions because the machine is in either an accepting or a rejecting state, depending on which state it is in, for a string of length 0.

2. As a partition is refined by identifying distinguishable states, eventually there must be a $(k+1)$ -equivalence partition which is exactly the same as the k -equivalence partition. This reasoning follows because the state set is finite, and there are therefore a limited number of times a boundary can be introduced into a partition. For this k , the k -equivalent states must be equivalent (with no string length restriction), since no further refinement is possible. This partition is $(k+1)$ -equivalent, $(k+2)$ -equivalent, etc. Therefore each subset of this partition is a set of equivalent states.

3. A partition is refined by noting whether two states in the same subset can be distinguished by some single input token. For example, suppose P and Q belong to the same subset in a k -equivalent partition, and we find (by examining the state transitions), that P goes to P' and Q goes to Q' on some token "b". If P' and Q' are in different partitions, then they are distinguishable; consequently P and Q must be distinguishable and belong in different subsets in the $(k+1)$ -equivalent partition. The two states may also be distinguished if one of them possesses a transition on some token, while the other does not. A nonexistent transition on a state P and token "b" means that the machine cannot scan string "b" in state P. If state Q has a transition on token "b" while P does not, then P and Q are distinguishable and belong in different subsets of the partition.

Summarizing, we begin with the two-fold partition of halt and non-halt states. Then we induce refinements on these by looking for single tokens that can distinguish two members of a subset. When no further refinements can be made, the machine has been reduced to minimal form.

Example. Consider the DFSA whose state transition table is given in figure 3.18.

The initial partition, on halt and nonhalt states, is

$$\{A, B, C, D\} \{E, F\}$$

We now attempt to refine this partition by looking for tokens that can distinguish pairs of states within either of the subsets.

Consider the pair (A,B). State B has a transition (to C) on “a”, but A does not; hence these states belong in different subsets.

Next consider the pair (A,C). Again, C has a transition on “a”, but A does not; hence, these belong in different subsets. Note that these conclusions do not prove that B and C belong in different subsets; hence, we must also consider pair (B, C).

States B and C have transitions on each of the three tokens. On token “0”, they both transfer to E, on token “1”, they both transfer to D, hence neither of these tokens serves to distinguish them. On token “a”, state B goes to C and state C goes to B. Since B and C are in the same partition, token “a” also fails to distinguish them. We conclude that B and C belong in a common subset in the next partition.

Next consider states A and D, both clearly distinguishable. Also, since D has only one transition (on token “0”), it is distinguishable from B and C. Hence D belongs in its own partition.

δ	0	1	a
A	B	C	
B	E	D	C
C	E	D	B
D	F		
ⓔ		D	
ⓕ			

Figure 3.18. A machine to be reduced.

Finally, consider states E and F. State E has a transition on “1” not possessed by F, hence this pair is distinguishable.

Our 1-equivalent partition therefore looks like this:

$$\{A\} \{B, C\} \{D\} \{E\} \{F\}$$

There is only one subset that is a potential candidate for a partition, the {B, C} pair. A glance at the table shows that this partition cannot be refined. Hence these two states must be equivalent. The reduced machine has five states. State C may be renamed “B” wherever it appears. The reduced machine is in figure 3.19.

Another Example. Consider the decimal number machine in figure 3.17. Its initial partition is

$$\{S, A, G\} \{H, BCE, DG, DH\}$$

States S, A, and G are clearly distinguishable. Similarly, state BCE belongs in its own partition. However, what about H, DG, and DH? They all transfer to a common subset on token “d”, whether in this partition or the next one. The final partition is then:

$$\{S\} \{A\} \{G\} \{BCE\} \{H, DG, DH\}$$

and the triplet subset cannot be further partitioned. Hence the final machine has five states, with two halt states, as shown in figure 3.20, which may be compared with figure 3.1; these two machines are clearly isomorphic.

δ	0	1	a
A	B	B	
B	E	D	B
D	F		
$\odot E$		D	
$\odot F$			

Figure 3.19. Machine of figure 3.18 reduced.

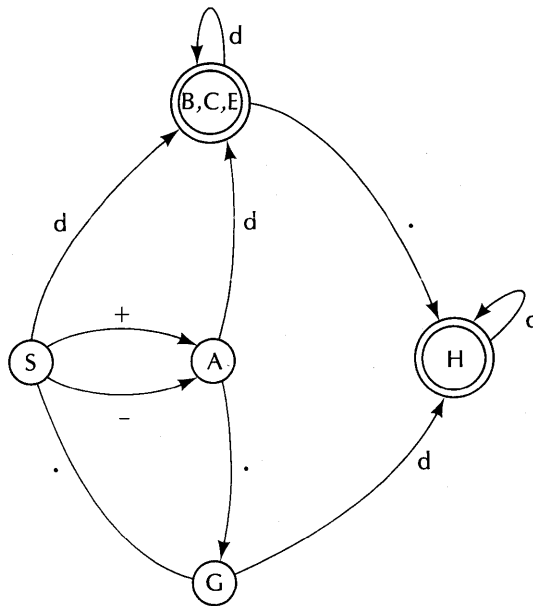


Figure 3.20. The machine of figure 3.12 made deterministic and reduced.

Summary

Recall that we asserted that the machine of figure 3.1 was equivalent to the nondeterministic machine of figure 3.3. We have now demonstrated that assertion, through the following machine transformation steps. These steps provide a systematic way to construct a program to recognize an important class of languages.

- Removal of empty move cycles (if any).
- Removal of empty moves.
- Removal of nondeterminism.
- Removal of inaccessible states.
- Reduction by identifying and merging equivalent states.

Since this process may be applied systematically to any FSA, and we have assurance that it will always yield a machine with the minimum number of states, it is possible to determine whether two different-appearing machines are in fact equivalent. We merely reduce each of them by the above process, then test them for isomorphism. We leave the matter of testing isomorphism for an exercise.

Exercises

1. Reduce the following FSA:

state	input	
	0	1
A	B	C
B	E	F
C	A	A
D	F	E
E	D	F
F	D	E

2. Show that if two states P and Q are k -distinguishable for $k \geq 0$ then they are $(k + 1)$ -distinguishable.
3. A machine M has n states. Give a bound for the largest k such that some pair of states is $(k - 1)$ -distinguishable but k -equivalent.
4. Develop an algorithm that tests two reduced machines M and M' for isomorphism.
5. Develop an algorithm that tests two nonreduced machines M and M' for equivalence. *Note:* The obvious approach is to reduce each of them, then apply the solution to exercise 4. Is there a more direct approach?

3.3.3. A Systematic Reduction Method

Several systematic tabular methods for machine reduction exist. We describe one that can be programmed easily on any computer, called the *pairs table* method.

A *pairs table* contains a pair of states or a *null* at the intersection of each row and column. Each column is associated with an input token. Each row is associated with a *feasible state-pair*. The reduction consists of two algorithms, one that builds a pairs table for the nonreduced deterministic FSA and one that marks certain rows. At the conclusion of the second algorithm, each row that is not marked is associated with a pair of equivalent states; each row that is marked is associated with a pair of distinguishable (nonequivalent) states. Furthermore, all the equivalent states appear as unmarked state-pairs in the final table.

A *feasible state-pair* is a pair of states that conceivably could be equivalent upon a cursory examination of the FSA transition table. More precisely, a state-pair (p, q) is a member of the feasible state-pair set if (1) $\{p, q\}$ is a subset of either F or Q-F, i.e., both are halt states or both are non-halt states, and (2) for every input token "a", $\delta(p, a)$ is $\emptyset$ if and only if $\delta(q, a)$ is $\emptyset$.

By selecting only those state-pairs that satisfy these two conditions, we eliminate from further consideration all those pairs of states that are obviously distinguishable, e.g. (1) a halt state is distinguishable from a non-halt state, and (2) a state with a transition on some symbol is distinguishable from another without a transition on that symbol.

For example, consider figure 3.17. The state-pair (S, A) is not a feasible state-pair, because S has a transition to A on token "+", while A does not. The state-pair (G, H) is not feasible because H is a halt state, while G is not.

Thus the set of feasible state-pairs for the machine of figure 3.17 consists of the set:

$$\{(H, DG), (H, DH), (DG, DH)\}$$

None of the other state-pairs are feasible. Since this machine is rather trivial under the reduction algorithm, we choose a more interesting machine, given in figure 3.21. For this machine, the set of feasible pairs includes all the internal combinations of the sets:

$$\{1, 3\}, \{2, 5, 7\}, \{4\}, \text{ and } \{6\}$$

	δ	Input		
		a	b	c
	1	2	5	
	2	3	4	1
	3	5	2	
States	④ 4	3	2	1
	5	1	4	1
	6	1		1
	7	3	6	3

Figure 3.21. Another machine to be reduced.

Thus the feasible pairs are

$$\{1, 3\}, \{2, 5\}, \{2, 7\}, \{5, 7\}$$

States 4 and 6 do not appear in any feasible pairs, since they are distinguishable from all the other states.

We now describe the pairs table construction. Given a pair (p, q) associated with a row, then the table entry for token “a” is the pair (p', q') , where $p' = \delta(p, a)$, and $q' = \delta(q, a)$. That is, we simply list the pair of states to which (p, q) transfers on each of the input tokens. Note that by the feasible pair selection process, only those states either possessing or not possessing such transitions are in the pairs table; hence we will get either a new state-pair or a null entry. Also, the machine is deterministic, which means that $\delta(p, a)$ and $\delta(q, a)$ contain at most one state each.

The resulting pairs table for the machine in figure 3.21 is shown in figure 3.22(a).

A pair is unordered, e.g., the pair $(2, 5)$ is equivalent to the pair $(5, 2)$. To facilitate the recognition of such equivalences, the members of each pair are written in numeric order in figure 3.22. Thus the states $(1, 3)$ actually transfer to $(5, 2)$ on token “b”; however, the state-pair $(5, 2)$ is written $(2, 5)$. By the same reasoning, if a pair $(2, 5)$ appears in the feasible state-pair list, we do not include $(5, 2)$.

(a) Unmarked

		Input		
		a	b	c
Feasible state pairs	(1,3)	(2,5)	(2,5)	
	(2,5)	(1,3)	(4,4)	(1,1)
	(2,7)	(3,3)	(4,6)	(1,3)
	(5,7)	(1,3)	(4,6)	(1,3)

(b) Marked

		Input		
		a	b	c
Feasible state pairs	(1,3)	(2,5)	(2,5)	
	(2,5)	(1,3)	(4,4)	(1,1)
	✓ (2,7)	(3,3)	(4,6)	(1,3)
	✓ (5,7)	(1,3)	(4,6)	(1,3)

Figure 3.22. Pairs table for machine of figure 3.21.

The next operation is marking those state-pairs that are distinguishable. This is the marking rule: A state-pair in the set of feasible state-pairs is *marked* if there exists a transition to a state-pair (p, q) such that (1) p and q are different, and (2) (p, q) is either marked or not among the feasible state-pairs.

This operation is repeated until no more state-pairs can be marked. Thus in figure 3.22, state-pair $(1, 3)$ transfers to state-pair $(2, 5)$ on both tokens “a” and “b”; $(2, 5)$ is in the feasible state-pair set and is unmarked, hence $(1, 3)$ is not marked.

Consider the pair $(2, 5)$, the second row, which has transitions to pairs $(1, 3)$, $(4, 4)$, and $(1, 1)$. The transitions to $(4, 4)$ and $(1, 1)$ do not call for marking, nor does the transition to $(1, 3)$, since $(4, 4)$ and $(1, 1)$ are singlet pairs, and $(1, 3)$ is in the table, unmarked.

Pair $(2, 7)$ is marked, however, since there is a transition to pair $(4, 6)$, on token “b”, and $(4, 6)$ is not in the table. Similarly, pair $(5, 7)$ is marked.

Repeating the operation on pairs $(1, 3)$ and $(2, 5)$, we find that they still are not marked, since $(2, 7)$ and $(5, 7)$ are not among the state-pairs to which they transfer. Hence we conclude that these two pairs remain unmarked, and are therefore the equivalent state-pairs.

The pairs algorithm therefore indicates that the equivalent state-sets of the machine in figure 3.21 are:

$$\{1, 3\}, \{2, 5\}, \{4\}, \{6\}, \{7\}$$

Exercises

1. Reduce the following FSA by the pairs table method:

state	input	
	0	1
A	B	C
B	E	F
C	A	A
D	F	E
E	D	F
F	D	E

2. Show that the members of each feasible state-pair are 1-equivalent.

3.4. Regular Grammars and FSA

Regular grammars and finite-state automata have a very close correspondence. Given any FSA, a regular grammar may be constructed from it whose language is identical to that of the FSA, and given any regular grammar, an FSA may be constructed from it whose language is identical to that of the

grammar. We give the constructions for these assertions, with examples. The proofs are elementary.

FSA from a Regular Grammar

Recall that a regular grammar $G = (N, \Sigma, P, S)$ has productions in the set P of the form

$$\begin{array}{ll} A \rightarrow aB & \text{where } B \in N, \text{ and } a \in \Sigma \\ A \rightarrow b & \text{where } b \in \Sigma \end{array}$$

Given a grammar G , we construct a NDFSA $M = (Q, \Sigma, \delta, q_0, F)$ as follows:

The states of M are associated with the nonterminals of G , except for one additional state q' not in N . The halt-state set F is $\{q'\}$. The state q_0 is associated with the start token S in G . Then for every production $A \rightarrow aB$ in G , we add state B to the transition set $\delta(a, A)$. For every production $A \rightarrow b$ in G , we add state q' to the transition set $\delta(b, A)$.

Note that the alphabets of M and G are identical, and that M may be nondeterministic.

For example, consider the simple grammar G_2 , with productions:

$$\begin{array}{ll} S \rightarrow + N \\ S \rightarrow - N \\ S \rightarrow d N \\ S \rightarrow d \\ N \rightarrow d N \\ N \rightarrow d \end{array}$$

The language of this grammar will be recognized as the signed decimal numbers, where “d” is a decimal digit.

The machine M will contain the states $\{S, N, F\}$, where F is a new halt state.

Then the transitions of M are defined by the following transition function:

$$\begin{array}{ll} \delta(S, +) = N \\ \delta(S, -) = N \\ \delta(S, d) = N \\ \delta(S, d) = F \\ \delta(N, d) = N \\ \delta(N, d) = F \end{array}$$

The moves of M in recognizing a string x mimic the derivation of x in the grammar. For example, consider the string “-313”, or “-ddd” as it appears in the grammar. The moves are

S to N on “–”,
 N to N on “d”,
 N to N on “d”,
 N to F on “d”.

The derivation is

$$S \Rightarrow -N \Rightarrow -dN \Rightarrow -ddN \Rightarrow -ddd$$

We leave a proof that $L(G) = L(M)$ to the reader.

FSA to a Regular Grammar

Given a FSA M , a regular grammar G may be constructed from M , such that $L(M) = L(G)$, as follows. Let M be deterministic. Let $G = (N, \Sigma, P, S)$, where N , the nonterminals, correspond to the machine states Q , the alphabets of G and M are identical, S corresponds to the start state of the machine, and the productions in P are constructed as follows:

- If $\delta(A, a) = B$, then include production $A \rightarrow aB$ in P .
- If $\delta(A, a) = B$, where B is in the halt set F , then include production $A \rightarrow a$ in P .

The resulting grammar is clearly regular. Again, machine M mimics a derivation in G for some string x . We leave a proof that $L(M) = L(G)$ to the reader.

Exercises

1. Construct a regular grammar for the FSA of figure 3.21, for the reduced FSA.
2. Construct an FSA for the regular grammar G , given below. Reduce it and discuss the language $L(G)$ informally.

$A \rightarrow 0B \mid 1D$
 $B \rightarrow 0B \mid 1C \mid 1$
 $C \rightarrow 1E \mid 0D$
 $D \rightarrow 0D \mid 1E \mid 1$
 $E \rightarrow 1C \mid 0B$
 $F \rightarrow 0C \mid 1G \mid 0$
 $G \rightarrow 1E \mid 0F \mid 1$

3. Show that if a string x can be derived in a grammar G , then it is accepted by the FSA M constructed as above. (*Hint*—show by induction on the derivation length).

3.5. Regular Expressions and FSA

A regular expression is a compact way of representing a regular language. In addition to string elements, a regular expression uses three basic operations, *concatenation*, *alternation*, and *closure*. We have introduced each of these operations previously, in a less formal manner.

3.5.1. Definitions

Concatenation is an associative, noncommutative binary operation. The token for concatenation is juxtaposition, e.g., if E_1 and E_2 are two regular expressions, then E_1E_2 is the concatenation of the two. If E_1 and E_2 denote the sets of strings S_1 and S_2 , respectively, then E_1E_2 denotes the set

$$S = \{uv \mid u \in S_1 \text{ and } v \in S_2\}$$

Another way to express E_1E_2 is: Choose any string “u” in the set denoted by E_1 ; choose any string “v” in the set denoted by E_2 ; then the concatenated string “uv” is in E_1E_2 . Also, any string in E_1E_2 can be divided into a “u” prefix and a “v” suffix, where “u” is in the set denoted by E_1 and “v” in the set denoted by E_2 .

Alternation is an associative, commutative binary operation, represented by the symbol $|$ or $+$. If E_1 and E_2 are two regular expressions, denoting the sets of strings S_1 and S_2 , then $E_1 | E_2$ is a regular expression denoting $S_1 \cup S_2$, the union of S_1 and S_2 . We have introduced alternation previously; it is commonly used in the BNF representation of production rules.

Closure is a unary operation. If E is a regular expression denoting some set of strings S , then $\{E\}$ is a regular expression, called the *closure* of E , and represents the set of all possible strings formed by choosing members of S and concatenating them, together with the empty string ϵ . Thus the closure $\{E\}$ of a regular expression E is a compact way of writing the infinitely large regular expression

$$\epsilon \mid E \mid EE \mid EEE \mid EEEE \mid \dots$$

Closure may also be defined by the following set expression:

$$\{E\} = \{xY \mid x \in E \text{ and } Y \in \{E\}\} \cup \{\epsilon\}$$

This definition may appear circular, since the closure operation which is being defined appears in its own definition. Therefore let us explore this matter briefly. By the definition, the empty string ϵ is in $\{E\}$, which means that Y can be ϵ within the definition and, by the first half of the union, all those x 's in E are also in $\{E\}$. In other words, $\{E\}$ contains $E \mid \epsilon$. Thus Y may contain anything in $E \mid \epsilon$ and, by the first half of the union, all those xY 's such that x is in E are in $\{E\}$; therefore $\{E\}$ contains $EE \mid E \mid \epsilon$, etc.

Closure may also be represented by the token $*$ following the expression to be closed, thus $E^* = \{E\}$. If E consists of more than one token, it must be enclosed in parentheses when this notation is used.

Recall that we earlier stated that Σ^* represents the set of strings in the alphabet Σ . This statement is clearly consistent with the definition of closure.

Parentheses may be used freely in regular expressions as needed to keep the relative ordering of the operations clear. Closure is represented by a parenthesis structure $\{\}$ and does not require any precedence rule; it operates on the expression contained therein. By convention, concatenation has a higher precedence than alternation, therefore the regular expression

$$ab|cde$$

is interpreted

$$(ab)|(cde)$$

Closure represented by $*$ has a higher precedence than either concatenation or alternation. Thus

$$a | bc^*$$

is interpreted

$$(a) | (b(c)^*)$$

The elements of a regular expression are the tokens of an alphabet Σ , the empty token ϵ , and the null set $\emptyset$. A null-set regular expression denotes an empty set, containing neither ϵ nor any strings. The regular expression ϵ denotes a string set containing only the empty string.

The symbols for alternation, closure, etc. are called *metasymbols* and cannot be in the alphabet of the regular language. Of course, parentheses appear in many common languages, so that any practical implementation must deal with a potential conflict of metasymbols and the language alphabet. A choice of metasymbols might be provided, or a special quote character might be used to delimit alphabet symbols.

A Context-Free Grammar for Regular Expressions

The language of regular expressions is expressed by the following context-free grammar $G_r = (N, \Sigma, P, R)$, where

$$N = \{R, C, L\}; \Sigma = \{+, (,), \{, \}, a\}, \text{ and}$$

the production set P is

$$\begin{aligned} R &\rightarrow R + C && (\text{alternation}) \\ R &\rightarrow C \\ C &\rightarrow C L && (\text{concatenation}) \end{aligned}$$

$C \rightarrow L$	
$L \rightarrow (R)$	(parenthesizing)
$L \rightarrow \{ R \}$	(closure)
$L \rightarrow a$	(any token in the regular language)
$L \rightarrow \epsilon$	(empty string)

This grammar expresses the precedence of the operations, as well as the structural rules for regular expression formation. In the following discussion, we shall assume that any regular expression may be parsed and a derivation tree created for it.

Examples of Regular Expressions

The regular expression

$$(+ \mid - \mid \epsilon) d \{ d \}$$

represents the set of (possibly) signed numbers, where d represents the set of digits. This expression may be interpreted in English as follows:

E consists of a choice of “+,” “−,” or empty (ϵ) followed by a single digit, followed by any number (including zero) of digits.

The regular expression

$$(+ \mid - \mid \epsilon) (d \{ d \} . \{ d \} \{ d \} . d \{ d \}) (\epsilon (E (+ \mid - \mid \epsilon) d \{ d \}))$$

represents a floating point number. To see this, let us divide the expression into three parts as follows:

$$\begin{array}{ccc} (+ \mid - \mid \epsilon) & (d \{ d \} . \{ d \} \{ d \} . d \{ d \}) & (\epsilon (E (+ \mid - \mid \epsilon) d \{ d \})) \\ \text{I} & \text{II} & \text{III} \end{array}$$

Then part I represents an optional sign. Part II is the mantissa. It must contain a decimal point and at least one digit ahead or behind the decimal point. Part III represents an optional exponent, signaled by an “E” followed by an optional sign and at least one digit.

Note the use of precedence of concatenation over alternation in part II.

Exercises

1. Describe informally the languages of each of the following regular expressions and give some examples of strings in each one:

$$\begin{aligned} & (A + \{B\})ABB \\ & 00\{01+10+11\} + 11\{01+10+00\} \\ & (a+b)\{a+b+0\}b \end{aligned}$$

2. Construct a regular expression that represents:

- (a) A sequence of 0's and 1's such that the combination 11 appears exactly once.
- (b) Arithmetic expressions containing binary $+$, $-$, $*$, $/$, unary $-$, and one level of parenthesis nesting.
- (c) The set of Cobol identifiers, consisting of a sequence of letters, digits, and underline ($_$), starting with a letter.

3.5.2. Regular Expression Identities

Regular expressions satisfy a number of identities, which may be used to reduce the complexity of a regular expression or to prove that two regular expressions represent the same language. Unfortunately, there seems to be no systematic procedure for transforming regular expressions into standard forms, as in ordinary algebra or trigonometry.

Let A , B , and C be regular expressions. Then the following identities hold:

- (1) $A + B = B + A$ (commutivity of alternation)
- (2) $\{\emptyset\} = \epsilon$ (closure of an empty set is the null string)
- (3) $A + (B + C) = (A + B) + C$ (associativity of alternation)
- (4) $A(B C) = (A B)C$ (associativity of concatenation)
- (5) $A(B+C) = A B + A C$ (distributivity of concatenation over alternation)
- (6) $A \epsilon = \epsilon A = A$ (identity of concatenation)
- (7) $\emptyset E = E \emptyset = \emptyset$ (zero of concatenation)
- (8) $\{E\} = E + \{E\}$
- (9) $\{\{E\}\} = \{E\}$
- (10) $E + \emptyset = E$ (identity of alternation)

Most of these follow directly from the corresponding properties of the string sets represented by the regular expression. For example, the “ $+$ ” operator corresponds to a set union, which satisfies commutivity and associativity. Similarly, concatenation can easily be shown to be associative.

Identity 2 follows immediately from the definition of closure— regardless of what (if anything) is in a set E , $\{E\}$ contains ϵ .

Identity 8 follows immediately from the observation that every string in E is also in $\{E\}$, hence the union of these two is exactly $\{E\}$.

Exercises

1. Show that the following are identities through use of the ten identities listed previously.

$$\begin{aligned} E\{E\} &= \{E\}EE \\ E(\epsilon + \{E\}E) &= \{E\}E \\ B(A + AA) &= BAA + BA \\ \{\{A\}\{B\}\} &= \{A + B\} \end{aligned}$$

2. Prove each of the ten identities through the use of the equivalent set definitions.

3.5.3. Correspondence to FSA

We now demonstrate that, for every regular expression E denoting some language $L(E)$, there exists a FSA M such that $L(M) = L(E)$. The construction of the machine is particularly useful, since it is often more convenient to represent a language as a regular expression than as a FSA. We therefore need a systematic way to construct a recognizer for the language of a regular expression.

The machine we construct will be nondeterministic, in general, but may be reduced to a minimal deterministic machine by the methods of the previous sections.

The basic idea of the transformation is simple. We conceive a FSA that contains a start state S and one halt state F . Somehow, it recognizes a regular expression E , as diagrammed in figure 3.23(a). The square box containing “ E ” represents a set of states and transitions between states S and F .

Now suppose that E is the empty set $\emptyset$. Since $\emptyset$ is an empty language, which contains neither any string nor ϵ , the only possible machine for $\emptyset$ is the isolated S and F states, figure 3.23(b). This machine refuses to accept any string, including the empty string, since it can never reach the halt state.

Next suppose that E is the empty string ϵ . A recognizing machine for this language is shown in figure 3.23(c), which permits one transition—an empty move from S to F . This machine clearly exactly accepts the empty string. A string of length > 0 is rejected, since there are no other moves from either S or F .

If E is an alphabet token “ a ”, the machine of figure 3.23(d) exactly recognizes a string consisting of that token.

Now we consider some more sophisticated machines. If E is a parenthesized expression, $E = (E')$, a recognizer machine for E is clearly a recognizer for E' .

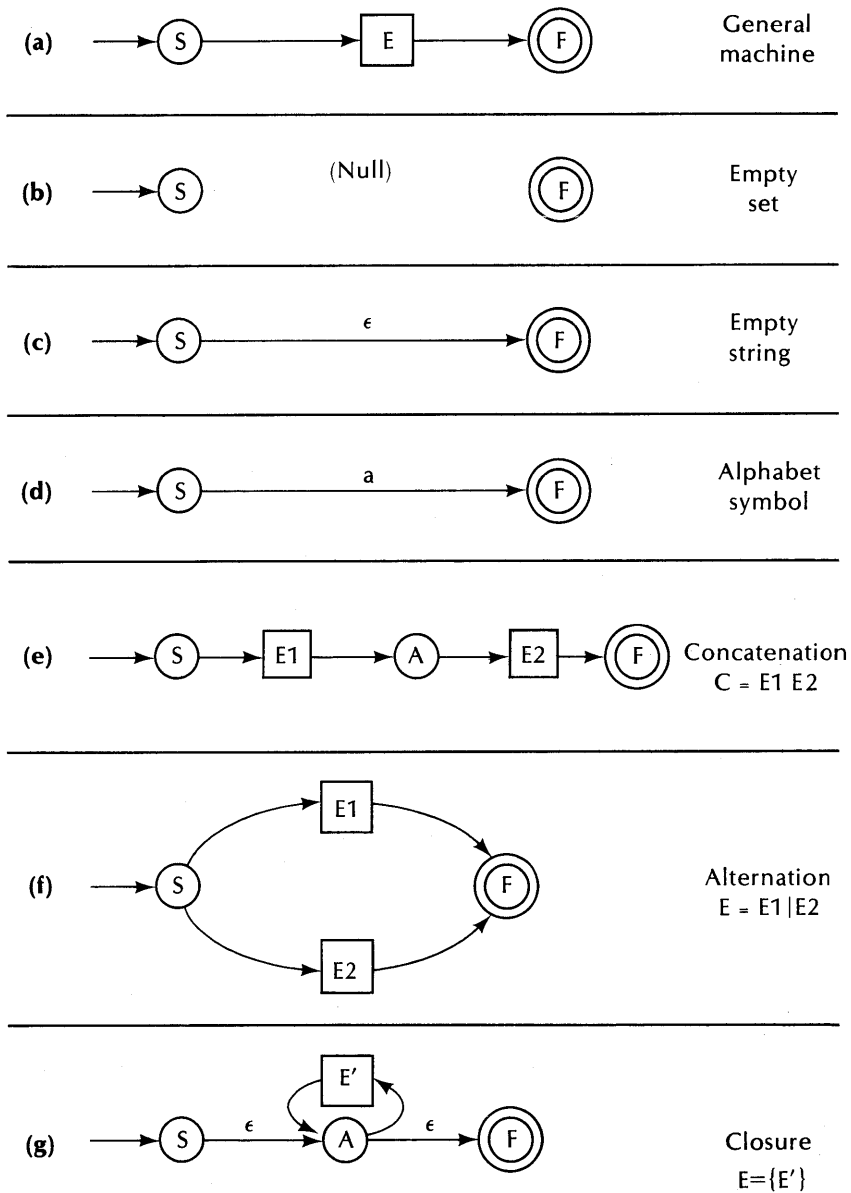


Figure 3.23. Generation of a non-deterministic finite-state automaton from the components of a regular expression.

Consider the concatenation E of two expressions, E_1 and E_2 . A recognizer for E clearly must recognize E_1 , then E_2 , in that order. Such a machine is shown in figure 3.23(e).

An alternation expression $E = E_1 \mid E_2$ is recognized exactly by the machine of figure 3.23(f). Recall that the set of strings represented by $E_1 \mid E_2$ is the union of the sets represented by E_1 and E_2 . Now let machine M_1 recognize E_1 and M_2 recognize E_2 . Then consider a string x in E . The string x must be in E_1 or in E_2 (it could belong to both, of course). If in E_1 , then the upper path of figure 3.23(f) yields a recognition; if in E_2 , then the lower path of figure 3.23(f) yields a recognition. No strings other than those in the union of $L(M_1)$ and $L(M_2)$ are in the composite machine; any string recognized by the composite would have to be recognized in the upper or lower path, therefore by either M_1 or M_2 .

Finally, a closure expression $E = \{E'\}$ is recognized exactly by the machine of figure 3.23(g). The empty moves are needed when these machine segments are combined to form a complete recognizer for some regular expression. Machine (g) clearly recognizes the empty string (two empty moves, S to A , A to F), and one or more concatenations of the strings in E' . Thus a string consisting of n members ($n \geq 0$) of the regular expression E' may be recognized by the empty move from S to A , n moves from A to itself through the machine for E' , followed by the move from A to F .

Figure 3.23 essentially outlines the rules by which a complete machine for an arbitrary regular expression may be built from its parts. The construction operations are effectively guided by the derivation tree for the regular expression, and may be done bottom-up or top-down. The top-down process will be illustrated for an example. Then a top-down procedure will be given.

Consider the regular expression

$$E = (+ \mid - \mid \epsilon)d\{d\}$$

which displays all the operations and tokens of regular expressions except the $\emptyset$ set, which should never appear within a regular expression anyway.

A simplified tree for this expression is given in figure 3.24. It is essentially the derivation tree with the single productions and parenthesis nodes removed.

At the root level, E consists of a concatenation of two pieces, which we shall call E_1 and E_2 :

$$E = E_1 E_2$$

where $E_1 = (+ \mid - \mid \epsilon)$ and $E_2 = d\{d\}$. Thus the first machine looks like figure 3.25(a). We have introduced a new intermediate state, A .

The transition from S to A is an alternation of $+$ with $(- \mid \epsilon)$, as shown in figure 3.24. The single transition is therefore split into two, one for “+”, the other for “ $(- \mid \epsilon)$ ”. The latter in turn splits into one for “-” and one for “ ϵ ”.

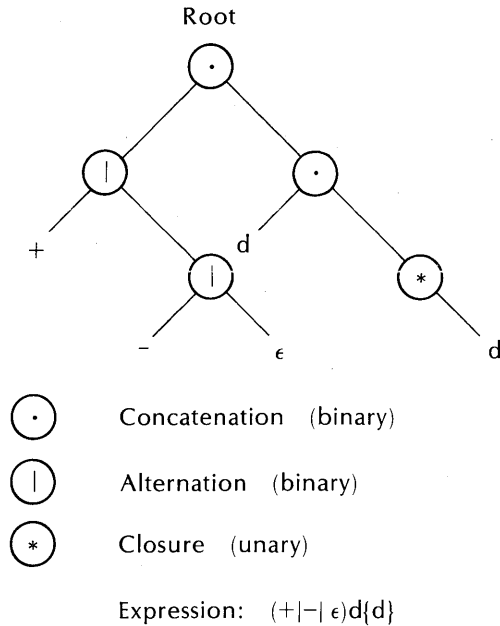


Figure 3.24. The regular expression “ $(+|-|\epsilon)d\{d\}$ ” as a tree.

The resulting machine is shown in figure 3.25(b).

Turning to the machine between states A and F, we see that it is another concatenation, of “d” with “{d}”. The machine of figure 3.25(c) is the result, containing another new state, B.

The last machine is between states B and F and is a closure machine. The final NDFSA for the regular expression is therefore shown in figure 3.25(d). As it turns out, neither of the ϵ -moves between states B and F are necessary in this machine; however, there is no harm in keeping them, since they will be eliminated and the machine reduced to minimal form by the methods previously described.

We now give a recursive procedure $\text{FSM}(\text{EXPR}, P, Q)$, which, when given a regular expression EXPR, an initial state $P = S$, and a final state $Q = F$, yields a set of states and a transition function for a machine (nondeterministic in general) whose language is that of the regular expression. The final machine has one halt state, F.

We use a Pascal-like notation for the procedure. The operator “=” is an equality comparison; operator “:=” means the left side’s value is replaced by the value of the right-side expression. Lowercase words represent keywords, and uppercase words represent variables and arguments. The “ $\cup$ ” is the set union operation. A comment starts with { and ends with }. The variable δ represents the tabular representation of the transition function. Thus

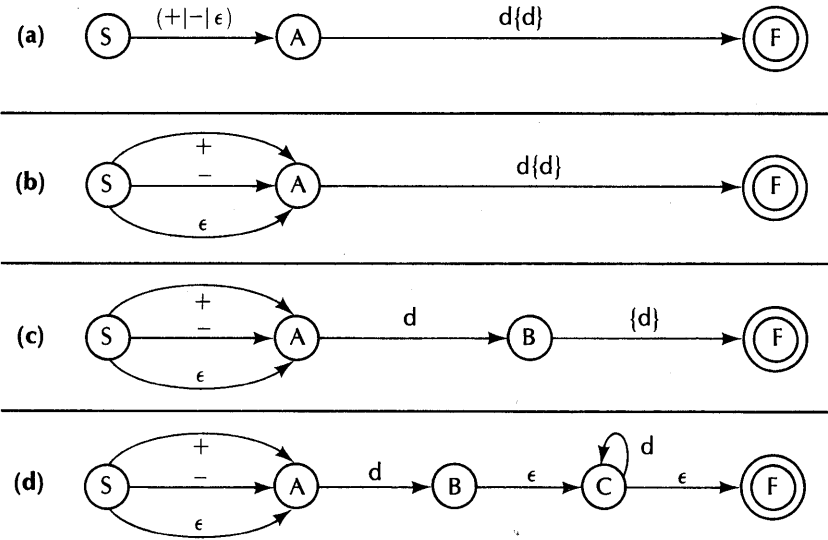


Figure 3.25. Development of a finite-state automaton from the regular expression $"(+|-|\epsilon)d\{d\}"$.

$$\delta(R, A) := \delta(R, A) \cup \{Q\}$$

means that state Q is added to the set $\delta(R, A)$.

```

procedure FSM(EXPR, P, Q);
begin
  if EXPR =  $\emptyset$  then return;
  if EXPR =  $\epsilon$  then
    begin
       $\delta(P, \epsilon) := \delta(P, \epsilon) \cup \{Q\}$ ;
      return;
    end;
  if EXPR =  $a \in \Sigma$ , then
    begin
       $\delta(P, a) := \delta(P, a) \cup \{Q\}$ ;
      return;
    end;
  if EXPR =  $(X)$  then
    begin
      call FSM(X, P, Q);
    end;
end;

```

```

    return;
end;

if EXPR = X Y then    {concatenation}
begin
    create a new state A;
    call FSM(X, P, A);
    call FSM(Y, A, Q);
    return;
end;

if EXPR = X | Y then  {alternation}
begin
    call FSM(X, P, Q);
    call FSM(Y, P, Q);
    return;
end;

if EXPR = {X} then    {closure}
begin
    create a new state A;
    call FSM( $\epsilon$ , P, A);
    call FSM(X, A, A);
    call FSM( $\epsilon$ , A, Q);
    return;
end;
end;

```

Example. Let $\text{EXPR} = (+ | - | \epsilon)d\{d\}$. Then we call $\text{FSM}(\text{EXPR}, S, F)$. S will be the start state and F the (only) halt state of the final machine for EXPR . The following is a trace of the calls of FSM , the actions taken, and the returns. The periods “...” indicate the depth of nesting in the recursive calls.

```

call FSM( '(+ | - |  $\epsilon$ )d{d}', S, F);
. create a new state:  $R_1$ ;
. call FSM( '(+ | - |  $\epsilon$ )', S,  $R_1$ );
. . call FSM( '+ | - |  $\epsilon$ ', S,  $R_1$ );
. . . call FSM( '+', S,  $R_1$ );
. . . .  $\delta(S, +) := \delta(S, +) \cup \{R_1\}$ ;
. . . . return;
. . . call FSM( '- |  $\epsilon$ ', S,  $R_1$ );
. . . . call FSM( '-', S,  $R_1$ );
. . . . .  $\delta(S, -) := \delta(S, -) \cup \{R_1\}$ ;
. . . . . return;

```

```

. . . . call FSM('ε', S, R1);
. . . . δ(S, ε) := δ(S, ε) ∪ {R1};
. . . . return;
. . . . return;
. . . return;
. . return;
. call FSM('d{d}', R1, F);
. . create a new state: R2;
. . call FSM('d', R1, R2);
. . . δ(R1, d) := δ(R1, d) ∪ {R2};
. . . return;
. . call FSM('{d}', R2, F);
. . . create a new state: R3;
. . . call FSM('ε', R2, R3);
. . . . δ(R2, ε) := δ(R2, ε) ∪ {R3};
. . . . return;
. . . call FSM('d', R2, R3);
. . . . δ(R2, d) := δ(R2, d) ∪ {R3};
. . . . return;
. . . call FSM('ε', R2, F);
. . . . δ(R2, ε) := δ(R2, ε) ∪ {F};
. . . . return;
. . . return;
. . return;
. return;

```

The result of this FSM call is the transition function shown graphically in figure 3.25(d).

Exercises

1. Show that the regular expression

$$(+ \mid - \mid \epsilon) (d\{d\} \mid d\{d\}.\{d\} \mid \{d\}.d\{d\})$$

yields the FSA of figure 3.3.

2. Add an exponent part to the expression of the previous expression, develop its FSA and reduce it.
3. Define data structures for procedure FSM in Pascal and develop a complete program around it. Assume that a reduced tree structure for the regular expression is available, similar to that in figure 3.24. (The next chapters deal with the problem of parsing a regular expression and generating a reduced tree from the parse).

3.5.4. Regular Expression of a Regular Grammar

We first observe that a given regular language has many representations in regular expressions, and that it is not easy to reduce a given regular expression to some minimal form. The method given below always yields a valid regular expression, but it may be larger than another one that also represents the same language. We know of no systematic reduction process for regular expressions similar to those useful in ordinary algebra.

We first introduce the concept of a *regular expression equation*, which contains regular expressions and variables $X[1]$, $X[2]$, etc., that stand for some unknown regular expression. These resemble linear equations and are written in standard form as follows:

$$X[1] = a[1,0] + a[1,1] X[1] + a[1,2] X[2] + \dots + a[1,n] X[n]$$

$$X[2] = a[2,0] + a[2,1] X[1] + a[2,2] X[2] + \dots + a[2,n] X[n]$$

·
·
·

$$X[n] = a[n,0] + a[n,1] X[1] + a[n,2] X[2] + \dots + a[n,n] X[n]$$

Each of the coefficients $a[i,j]$ is a regular expression in general, but contains no variables.

Note that a set of productions in a regular grammar may be represented as a set of regular expression equations. For example, the regular grammar

$$S \rightarrow 0A$$

$$S \rightarrow 1B$$

$$S \rightarrow 0$$

$$S \rightarrow 1$$

$$A \rightarrow 0S$$

$$A \rightarrow 1B$$

$$A \rightarrow 1$$

$$B \rightarrow 0A$$

$$B \rightarrow 1S$$

$$B \rightarrow \epsilon$$

may be written as a set of three regular expression equations in the three unknowns, S , A , and B , as follows:

$$S = (0+1) + 0A + 1B \tag{3.1}$$

$$A = 1 + 0S + 1B \tag{3.2}$$

$$B = \epsilon + 1S + 0A \tag{3.3}$$

Thus, the first four productions are equivalent to

$$S \rightarrow 0A \mid 1B \mid 0 \mid 1$$

which may be written, using “+” for the alternation “|” and “=” for “→”, as:

$$S = 0 + 1 + 0A + 1B$$

We then observe that if it is somehow possible to solve a system of equations for the variable S , the solution being a regular expression in the alphabet, we will have a regular expression representing the language of the underlying grammar.

We first need a solution for the equation

$$S = aS + b \quad (3.4)$$

where “ a ” and “ b ” are regular expressions in the alphabet and possibly in the other variables. Note that any regular expression equation in any variable S may be written in this form.

A solution for this equation is

$$S = \{a\}b$$

or $S = a^*b$, using the alternate notation.

To prove this, consider the substitution of $\{a\}b$ for S in equation (3.4):

$$\{a\}b = a\{a\}b + b \quad (3.5)$$

Factor the right side, yielding

$$\{a\}b = (a\{a\} + \epsilon)b$$

Now $\{a\} = a\{a\} + \epsilon$, since ϵ is in both sides, “ a ” is in both sides, and any string in $\{a\}$, other than “ a ” or ϵ , is in $\{a\}$ and in $a\{a\}$. Hence Eq. (3.5) is an identity, and $\{a\}b$ is a solution of Eq. (3.4).

We will not show that $\{a\}b$ is in some sense a complete solution of Eq. (3.4). Now there are solutions to (3.4) that are not in $\{a\}b$, if “ a ” contains the empty string. Indeed, $\{a\}(b+c)$ is a solution to (3.4), where c represents any set of strings whatsoever, if “ a ” contains the empty string. However, it turns out that $\{a\}b$, called the *minimal fixed point* of Eq. (3.4), is sufficient to generate an equivalent regular expression.

Now we can solve a general system of equations for the start token S . We illustrate the method using Eqs. (3.1) to (3.3) given above. The general method should be clear from this example; a more rigorous treatment is given in Aho[1972].

We start with some equation other than the S equation, for example, the B equation, Eq. (3.3). If this had the form

$$B = aB + b$$

we would first transform it into the equation

$$B = \{a\}b$$

which eliminates B from the right-hand side. Since the right side of (3.3) does not contain B , this step is unnecessary.

The regular expression obtained for B , which is just Eq. (3.3), may now be substituted into the other equations. The resulting equations are then free of variable B . The result of this substitution in the set (3.1) and (3.2) is

$$\begin{aligned} A &= 1 + 0S + 1(\epsilon + 1S + 0A) \\ &= 1 + (0 + 11)S + 10A \end{aligned} \quad (3.5)$$

$$\begin{aligned} S &= (0 + 1) + 0A + 1(\epsilon + 1S + 0A) \\ &= (0 + 1) + (0 + 10)A + 11S \end{aligned} \quad (3.6)$$

We have made use of some of the identities in the second step in each case. For example, in the A equation,

$$\begin{aligned} 1 + 0S + 1(\epsilon + 1S + 0A) &= 1 + 0S + 1\epsilon + 11S + 10A \\ &= 1 + 1 + 0S + 11S + 10A = 1 + (0 + 11)S + 10A \end{aligned}$$

in Eq. (3.5).

We next rewrite Eq. (3.5) in the form $A = aA + b$:

$$A = 10A + (1 + (0 + 11)S) \quad (3.7)$$

which has the minimal fixed point solution:

$$A = \{10\}(1 + (0 + 11)S) \quad (3.8)$$

Substituting this solution into the remaining S equation yields:

$$S = (0 + 1 + (0 + 10)\{10\}1) + ((0 + 10)\{10\}(0 + 11) + 11)S \quad (3.9)$$

after some rearrangement and factoring. It has the fixed-point solution

$$S = \{(0 + 10)\{10\}(0 + 11) + 11\}(0 + 1 + (0 + 10)\{10\}1) \quad (3.10)$$

which should be a regular expression equivalent to the original regular grammar given above. As a check, it would be wise to construct an automaton from Eq. (3.10), reduce it, and verify that its regular grammar agrees with the original grammar. It is clearly not obvious that Eq. (3.10) reflects our grammar, nor is it clear whether a shorter expression can be found for S . Different expressions result, depending on which variables are eliminated first, and it may pay to do the reduction in different ways to see if a shorter expression can be obtained.

Exercises

1. Transform the FSA of figure 3.25(d) into a regular grammar, then into a regular expression. Can you show that the result is equivalent to $(+ | - | \epsilon)d\{d\}^*$ (Note: Empty moves must be removed first.)
2. Solve the following set of regular expression equations:

$$\begin{aligned} A &= (\{0\} + 1)A + B \\ B &= 11 + 0A + 11C \\ C &= \epsilon + A \end{aligned}$$
3. Show that $\{a\}^*(b+c)^*$ is a solution of $S = aS + b$ if “a” contains ϵ , and “c” is any string whatever.

3.6. FSA Representations

A deterministic FSA may be embedded in a computer program in either of two ways—as a set of tables which are interpreted by a general purpose program, or as a specially constructed program that represents the machine. The table approach is usually superior to the program approach for large automata in memory and in reliability. The table interpreter need be written only once for any machine whatever, and usually a machine table requires less storage space than the equivalent program instructions. However, interpretation of a table demands more running time. Therefore, if minimal run-time without regard to storage space is wanted, a program approach is better. If minimal storage space is wanted, then the table approach is better.

Interpreted Tables

A FSA table is based on its transition table. Usually, semantic actions on certain transitions are also needed, hence each transition should carry an additional table entry that specifies a semantic action.

FSA tables are generally sparse. By this we mean that most of the table entries are empty. For a large machine, over 90 percent of the table may consist of empty entries. We therefore have an opportunity to construct a set of tables that specify only the useful entries, omitting the empty entries. We now describe how this might be done.

3.6.1. Sparse Array Tables

The fundamental idea of a sparse array table system is rather simple, although the implementation looks complicated. We collect all the non-

NULL table entries together in order into one linear array, called VALUEA. Let its size be T ; this is the total number of non-NULL entries in the original table. Then we create another array of size T . It carries one of the index values associated with the values in VALUEA, say the J indices. Call this array INDEXJ. Now suppose the value V (non-NULL) is in the original array at position (I,J) , and shows up in the VALUEA array at index K . Then $\text{INDEXJ}(K) = J$.

We now need a guide to the I index values in VALUEA and INDEXJ, and it is provided by two other smaller arrays, INDEXI and NUMBEROFJ. Given an index I , INDEXI(I) is the VALUEA and INDEXJ index of the list found in row I of the original matrix. NUMBEROFJ(I) is the number of values in that list.

For example, consider the following matrix $A(I,J)$:

J \ I	1	2	3	4	5
1			15	9	
2	3				7
3		5			
4				10	

We first list its values in the array VALUEA, working left-to-right then down:

K	VALUEA
1	15
2	9
3	3
4	7
5	5
6	10

Next, we add the associated INDEXJ values:

K	VALUEA	INDEXJ
1	15	3
2	9	4
3	3	1
4	7	5
5	5	2
6	10	4

The last tables are INDEXI and NUMBEROFJ, as follows:

I	INDEXI	NUMBEROFJ
1	1	2
2	3	2
3	5	1
4	6	1

Then, to find $A(2, 5)$, we enter INDEXI(2) and NUMBEROFJ(2), yielding 3 and 2, respectively. Then enter INDEXJ(3) and search the list for at most 2 items; these are 1 and 5. Since 5 matches J in $A(I, J)$, the associated VALUEA is 7, which is $A(2, 5)$.

The algorithm is expressed by the following Pascal procedure, that returns the value $A(I, J)$, given the array declarations described above:

```

function A(I, J: integer): integer;
begin
    var K: integer;
    if NUMBEROFJ(I)=0 then A:=0 {indicates empty}
    else
    begin
        for K:=INDEXI(I) until
            INDEX(I)+NUMBEROFJ(I) do
        begin
            if J=INDEXJ(K) then
            begin
                A:=VALUEA(K);
                return
            end
        end;
        A:=0
    end
end
end

```

Suppose an array A has the dimensions (M, N) , and it contains T useful (nonempty) elements. Then a two-dimensional array would require $M \times N$ entries, while the reduced sparse matrix system defined above would contain $2(M + T)$ entries. There is a considerable saving in storage space with the sparse scheme for large M and N , and T much smaller than $M \times N$. For example, if $M=20$, $N=100$, and $T=20$, then $M \times N = 2000$, while $2(M + T) = 80$, a reduction in table size of 25 to 1. Furthermore, the NUMBEROFJ array usually consists of fairly small numbers, which may possibly be compacted in memory as bytes, which the direct storage of an array A requires $M \times N$ units of whatever storage is required for the A values.

Clearly, either of the dimensions in $A(I, J)$ may be chosen for the INDEXI table. The sizes of the INDEXJ and VALUEA tables are not affected by the choice, but the INDEXI and NUMBEROFJ tables are. The index (I or J) with the least cardinality should therefore be selected for the INDEXI and NUMBEROFJ table index.

3.6.2. Table Reductions

The NUMBEROFJ table is often unnecessary. If INDEXI is arranged in monotonic increasing order, we may infer the NUMBEROFJ value from two consecutive INDEXI values. That is,

$$\text{NUMBEROFJ}(I) = \text{INDEXI}(I+1) - \text{INDEXI}(I)$$

which is valid for all but the largest I. We therefore need one more entry in INDEXI, for index 5, and the NUMBEROFJ table may be dispensed with. The resulting INDEXI table from the previous example then looks like this:

I	INDEXI
1	1
2	3
3	5
4	6
5	7

The INDEXJ list may also be arranged in monotonic increasing order. The storage space required for it may be reduced appreciably in one of two ways. Suppose the smallest unit of storage is an element whose maximum value is V. Then the state table may be listed as actual states, until the state number exceeds V. At that point, all of the subsequent numbers are reduced by V, and started over. A special marker is needed to indicate the “breaks” in the table, and the table search algorithm must be organized to reflect this change.

Another way to reduce the INDEXJ list is to record only the positive increments from one state to the next. Except for the first state, which could be a fairly large number, most of the subsequent states are likely to be small numbers. This plan breaks down if the table contains a few large increments.

3.6.3. Sparse Array Representation of a FSA

A FSA is clearly expressed by its transition function, which is just a two-dimensional array, $\delta(p, a)$. Following the guidelines given above, we should choose the least dimension. Usually the token set is smaller than the state set, particularly if letters can be lumped together into one token and numbers can be lumped together into one token, for transition purposes.

Then the total number of tokens may be a few dozen at most. The number of states can be much larger, and depends on the complexity of the machine's language.

We also need some way to indicate semantic actions and whether the machine may halt in a given state. We are therefore led to the following set of tables for a general FSA:

READX, size = number of tokens. For a token I , **READX**(I) is an index into the **STATE** table.

STATE, size = number of transitions. Let $K = \text{READX}(I)$, for an input token I , and $N = (\text{READX}(I + 1) - \text{READX}(I))$. Then the list **STATE**(K), **STATE**($K + 1$), ..., **STATE**($K + N - 1$) contains all the legal present states corresponding to the input token I . If a match with the present state S cannot be found, then the machine blocks (error). If a match can be found, let the **STATE** index be M , i.e., **STATE**(M) = present state.

CALL, size = number of transitions. **CALL**(M), where M is found as explained in the preceding paragraph, contains information on which semantic action to perform. In general, **CALL**(M) will be a number specifying one of several possible semantic actions, a branch address, or a subroutine location.

HALT, size = number of transitions. This array contains single bits, T or F . If **HALT**(M) = T , then a halt is legal on this state and input token; otherwise, a halt is illegal (error if this is the last token).

GOTO, size = number of transitions. This array contains the next state; i.e., if the current state P and token "a" result in the **STATE** index M , then **GOTO**(M) = $\delta(P, a)$. The next state is used if another input token exists; otherwise, the machine halts and reports "error" or "accept", depending on the **HALT** bit.

Example FSA Expressed as a Sparse Table

Figure 3.26 shows the FSA of figure 3.17 expressed as a sparse matrix system. (State **DH** is equivalent to state **DG**, and is therefore not included.) The states are numbered:

S:	1
A:	2
G:	3
H:	4
BCE:	5
DG:	6

Index Symbol State
index

1	(+)	1
2	(-)	2
3	(.)	3
4	(d)	6
5		12

READX

Index	State	Call	Halt	GOTO
(+) 1	1 (S)	1	F	2
(-) 2	1 (S)	2	F	2
(.) 3	1 (S)	3	F	3
(.) 4	2 (A)	4	F	3
(.) 5	5 (B,C,E)	5	T	6
(d) 6	1 (S)	6	F	5
(d) 7	2 (A)	7	F	5
(d) 8	3 (C)	8	F	4
(d) 9	4 (H)	9	T	4
(d) 10	5 (B,C,E)	10	T	5
(d) 11	6 (D,G)	11	T	6

Figure 3.26. Finite-state automaton expressed as a sparse matrix table, for machine of figure 3.17.

The input tokens are also numbered:

+: 1
-: 2
.: 3
digit: 4

For example, consider input token “digit” (4), and present state H (4). The READX table says that the list of legal current states begins at index 6 in STATE table, and the list contains 6 elements. We therefore search the STATE table from index 6 through 11, and find state 4 at index 9. Corresponding to this index is a semantics operation call (9), a “T” HALT indicator (legal to halt if this is the last token), and GOTO=4 (next state is number 4, H).

Exercises

1. Trace the acceptance of the string
 $+dd.d$
 through the sparse table system of figure 3.26.
2. Write a Pascal procedure that interprets a set of reduced FSA sparse tables and calls an action procedure on every transition with a semantics action indicated in the table. It accepts tokens from a scanner.
3. Write a program that generates reduced tables from a complete state table.

3.6.4. Program Representation of a FSA

A direct program representation of a FSA will be the most time-efficient means of representation, if memory size is no consideration. There are many forms of representation, and the best will likely depend on the kind of machine instructions available. If the machine contains an instruction that can search for a given number in a list of numbers, returning its index, then essentially it can solve the problem of searching the STATE list in the sparse tables described above, and may also be used effectively in a direct program representation.

For example, the following program represents the FSA of figure 3.1. The states S, A, B, G, and H are represented by numbers 0, 1, 2, 3, and 4. It contains several utility procedures, as follows:

OPEN'INPUT, which opens the input list file, preparing it for reading by procedure NEXT'CHAR.

NEXT'CHAR, which fetches the next character, placing it in CHARACTER, returning TRUE if a character exists or FALSE if the input list is exhausted.

ERROR, which reports a machine block or failure to terminate in a halt state. What happens after an error is detected is not defined in this program. We may assume for now that the error is simply reported, and the program halts.

FSA Program Example

```
STATE:=0;
OPEN'INPUT;
```

```

while NEXT'CHAR do
begin
  case STATE of
    0: if "0" ≤ CHARACTER ≤ "9" then STATE:=2
        else if CHARACTER = "+"
            or CHARACTER = "-" then
            STATE:=1
        else if CHARACTER = "." then STATE:=3
        else ERROR;

    1: if "0" ≤ CHARACTER ≤ "9" then STATE:=2
        else if CHARACTER = "." then STATE:=3

    2: if "0" ≤ CHARACTER ≤ "9" then STATE:=2
        else if CHARACTER = "." then STATE:=4
        else ERROR;

    3: if "0" ≤ CHARACTER ≤ "9" then STATE:=4
        else ERROR;

    4: if "0" ≤ CHARACTER ≤ "9" then STATE:=4
        else ERROR;

  end;
end;    {end of while-do loop}

      {if here, the input list is empty}

if not(STATE=2 or STATE=4) then ERROR;

```

Summary

The choice between a direct program representation or an interpreted table approach to a FSA depends on the size of the FSA, the desired efficiency, and the size of the available memory. For a small FSA, containing perhaps fewer than a dozen or so states, a direct program representation would probably be more efficient and require less computer memory, since the interpreting program is fairly complicated, and is no different in memory size for a large FSA than for a small one.

For a large FSA, a direct program should be attempted only if an automatic program generator is available. The possibility of an error increases with the number of transitions, and the program itself becomes less and less readable. Maintenance is also a problem, since a minor change in the FSA may require a major rewriting of the program and all the state numbers.